Bausteine Computergestützter Datenanalyse

Lukas Arnold Simone Arnold Florian Bagemihl
Matthias Baitsch Marc Fehr Franca Hoillmann
Maik Poetzsch Sebastian Seipel

2026-03-03

Inhaltsverzeichnis

Werkzeugbaustein Python **5**
Voraussetzungen . . . 5
Lernziele . . . 5

1 Introduction: Data Analysis with Python **7**
1.1 Fundamentals of Object-Oriented Programming . . . 8
1.1.1 Classes, Types, Objects, Attributes . . . 8
1.2 Formatting program code . . . 10
1.3 Formatting Output . . . 12
1.4 Integers . . . 13
1.5 Floating-Point Numbers . . . 13
1.6 Strings . . . 14
1.7 Exercises . . . 14

2 Data Types **17**
2.1 Numbers . . . 17
2.1.1 Integers . . . 17
2.1.2 Floating-Point Numbers . . . 18
2.2 Arithmetic Operators . . . 19
2.3 Number Exercises . . . 20
2.4 Boolean Values . . . 22
2.5 Logical Operators . . . 23
2.6 Boolean Value Exercises . . . 24
2.7 Strings . . . 26
2.8 String Operations . . . 27
2.9 String Exercises . . . 27
2.10 Variables . . . 28
2.10.1 Additional Assignment Operators . . . 30
2.10.2 Naming Variables . . . 30
2.11 Variable Exercises . . . 31

3 Functions: Basics **34**
3.1 Functions and Methods . . . 35
3.1.1 Functions . . . 35
3.1.2 Methods . . . 36

3.2 Parameters . . . 38
3.3 Function Exercises . . . 40

4 Control Flow **42**
4.0.1 Branching . . . 42
4.0.2 Loops . . . 44
4.0.3 Exception Handling . . . 49
4.1 Control Flow Exercises . . . 52

5 Collection Types **56**
5.1 Lists . . . 56
5.1.1 Slicing: the Access Operator [] . . . 57
5.1.2 List Methods . . . 59
5.1.3 List Exercises . . . 63
5.2 Tuples . . . 65
5.2.1 Copying Tuples . . . 66
5.3 Sets . . . 67
5.3.1 Copying Sets . . . 68
5.4 Dictionaries . . . 68
5.4.1 Copying Dictionaries . . . 69
5.5 Overview of Collection Types . . . 70
5.6 Deleting: the `del` Keyword . . . 71
5.7 Functions . . . 71
5.8 Operations: Using Loops . . . 72
5.9 Collection Types Exercises . . . 72

6 Defining Your Own Functions **76**
6.1 Syntax . . . 76
6.2 Optional Parameters . . . 77
6.3 Return Value(s) . . . 79
6.4 Exercises Defining Functions . . . 79
6.5 Using Lists . . . 82
6.6 Using ord() and chr() . . . 84

7 Reading and writing files **87**
7.1 File objects . . . 87
7.1.1 File path . . . 87
7.1.2 Access mode . . . 88
7.1.3 Print File Content . . . 91
7.2 Reading Files . . . 91
7.3 Task: Reading Files . . . 94
7.4 Interpreting Data . . . 95
7.5 for-loop with break . . . 96

7.6 Method `file_object.readline()` 96
7.7 Task: Interpret Data 100
7.8 Reading as a List 102
7.9 Writing Files 104
7.10 Task: Writing files 104

8 Importing Modules and Packages 105
8.1 import as 107
8.2 Small Module Overview 108

9 The Essentials 109

Werkzeugbaustein Python

Bausteine Computergestützter Datenanalyse von Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch und Sebastian Seipel. Werkzeugbaustein Python von Marc Fehr und Maik Poetzsch ist lizensiert unter CC BY 4.0. Das Werk ist abrufbar auf GitHub. Ausgenommen von der Lizenz sind alle Logos Dritter und anders gekennzeichneten Inhalte. 2025

Zitiervorschlag

Arnold, Lukas, Simone Arnold, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, und Sebastian Seipel. 2025. „Bausteine Computergestützter Datenanalyse. Werkzeugbaustein Python“. https://github.com/bausteine-der-datenanalyse/w-python.

BibTeX-Vorlage

```
@misc{BCD-w-python-2025,
 title={Bausteine Computergestützter Datenanalyse. Werkzeugbaustein Python},
 author={Arnold, Lukas and Arnold, Simone and Baitsch, Matthias and Fehr, Marc and Hollmann,
 year={2025},
 url={https://github.com/bausteine-der-datenanalyse/w-python}}
```

Voraussetzungen

No prerequisites; the Pseudocode Tool Module is helpful.

The estimated time to complete this module is approximately **placeholder**.

Lernziele

In this module, the fundamentals of programming with Python are introduced. In this module, you will learn to…

- Become familiar with the basic concepts of object-oriented programming.

- Write Python code, create variables, perform operations, and format output.
- Read and use documentation.
- Understand Python's exclusive counting system (zero-based indexing).
- Know the difference between functions and methods and how to write your own functions.
- Load modules and packages.

1 Introduction: Data Analysis with Python

Generating and analyzing data is a central part of scientific research. Computer-based data analysis allows large datasets to be evaluated (partially) automatically. Readable scripting languages like Python ensure transparent data processing and enable analyses to be repeated or adapted "at the push of a button."

Abbildung 1.1: Logo of the Python Programming Language

Python logo by the Python Software Foundation is licensed under GPLv3. The wordmark is trademarked: https://www.python.org/psf/trademarks/. The asset is available on Wikimedia. 2008

Python appears as a simple console. Python code can be entered into the console or saved in a plain text file called a script. The program code is executed by an interpreter, which translates the script's instructions into machine code for the specific computer system. This allows the script to run on different systems. Modern Python interpreters are not limited to ASCII characters and can also handle characters in UTF-8, which extends ASCII to include special characters, such as German special characters.

Many features, such as code formatting, autocomplete, and error checking, are provided by an Integrated Development Environment (IDE).

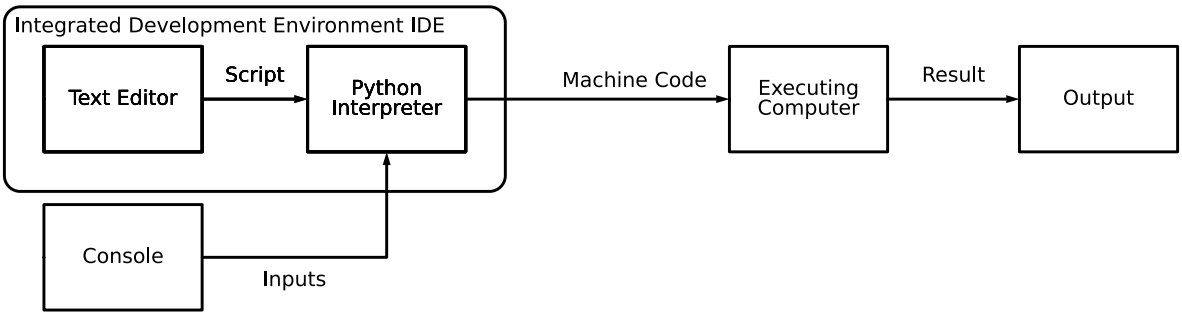


Abbildung 1.2: Software development with Python

1.1 Fundamentals of Object-Oriented Programming

Python is an object-oriented programming language. Object-oriented programming is a system for bringing order to complex computer programs. In this section, the basic concepts of object-oriented programming with Python are introduced. You will learn the difference between an object, a class, and a data type.

1.1.1 Classes, Types, Objects, Attributes

A Python program consists of various elements: operators and operands, functions and methods, values and variables, and much more. Everything in Python is an object.

Every object belongs to a class, for example, the class of integers. The class acts as a blueprint defining the *properties* and *behavior* of the object—such as which data it can store and which operations can be performed. A brief example: depending on their class, objects behave differently with the `+` operator.

```
print(type(2), 2 + 2, "Integers are added.")
print(type('a' and '2'), 'a' + '2', "Strings are concatenated.")
print(type(True), True + True, "Boolean values are added.")
```

```
<class 'int'> 4 Integers are added.
<class 'str'> a2 Strings are concatenated.
<class 'bool'> 2 Boolean values are added.
```

This is because the behavior of the `+` operator is defined for the classes integers (`int`), strings (`str`), and Boolean values (`bool`).
It behaves differently with `None`, a class used to represent non-existent values:

```python
print(None + None)
```

```
unsupported operand type(s) for +: 'NoneType' and 'NoneType'
```

Python has many classes. In Python, classes (`class`) are also called types (`type`). In earlier versions of Python, classes and types were still different. Nowadays, this distinction no longer exists, but both terms are still used in the language.

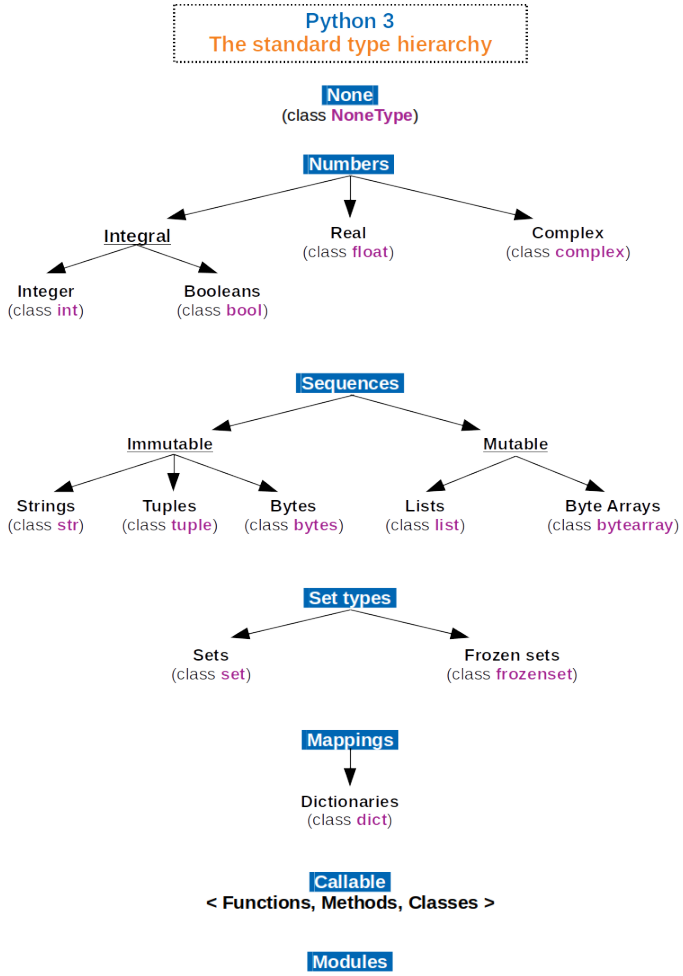


Abbildung 1.3: Data types in Python

Python 3. The standard type hierarchy. by is licensed under CC BY SA 4.0 and available on Wikimedia. 2018

Which class or type an object belongs to can be determined using the `type()` function.

```
print(type(print))
```

```
<class 'builtin_function_or_method'>
```

Attributes store properties of an object. They appear in the form `object.attribute` and are accessed without following parentheses. At this stage of the introduction, attributes have no practical relevance, but we will encounter them again later. A second form of attributes is the method. Methods are functions that belong to a specific class. Methods have the form `object.method()`, meaning they are called with following parentheses. We will learn how to use functions and methods in the upcoming chapters. How to determine the available attributes and methods of an object is explained in Beispiel 2 and Beispiel 1.

1.2 Formatting program code

When formatting Python code, only a few points need to be considered. To produce output in Python, the function `print(input)` is used. This function outputs the argument `input`.

1. Numbers and operators can be entered directly. Text, more precisely a string, must be enclosed in single or double quotation marks; otherwise, Python interprets it as the name of an object. Strings can contain letters, special characters, and numbers.

```
print(1 + 2)
print('123: Hello World!')
greeting_text = 'Hello Python!'
print(greeting_text)
```

```
3
123: Hello World!
Hello Python!
```

2. Comments are marked with a preceding hash `#`. Comments indicate code that should not be executed or provide explanations.

```
# A pure comment
# print("Python is great!") # commented-out code, followed by a comment
print("Python is pretty good.") # executable code, followed by a comment
```

```
Python is pretty good.
```

3. Expressions must be on a single line. Longer expressions can be continued over multiple lines using the backslash \ (there must be no # after \). Within functions such as `print()`, lines can be continued after each comma.

```
variable1 = 15
variable2 = 25

# Line continuation using \
total = variable1 + \
    variable2

# Line continuation inside a function
print(variable1,
      variable2,
      total)
```

```
15 25 40
```

In the example above, variables are created. We will deal with variables in the next chapter. However, I would like you to take a brief look at ***`variable1`*** *and* ***`variable2`*** *again. We will come back to them later.*

4. The number of spaces between operands and operators can be arbitrary.

```
print(1+0, 1 + 1, 1 +                  2)
```

```
1 2 3
```

5. Indentation with spaces indicates a cohesive code block. Within a code block, the same number of spaces must always be used. At least one space is required; beyond that, the number of spaces is flexible. Common conventions are 2 or 4 spaces.
   The following for-loop executes all statements in the indented execution block. The subsequent, non-indented line marks the beginning of a new statement that does not belong to the loop.

```
for i in range(2):
    print(variable1)
    print(variable2)
print(total)
```

```
15
25
15
25
40
```

1.3 Formatting Output

With so-called `f-strings`, you can create formatted strings. Formatted strings are created by prefixing the quotation marks with an `f`. Values and variables can be inserted using placeholders, which are indicated with curly braces `{}` and can include formatting instructions. The format specification inside the curly braces is simplified as follows:

```
{variable_name:width_type}
```

An f-string with placeholders without formatting information

```
number1 = 5
number2 = 7
ratio = number1 / number2
print(f"The ratio of {number1} to {number2} is {ratio}.")
```

```
The ratio of 5 to 7 is 0.7142857142857143.
```

The number of decimal places to display can be set as follows: `{ratio:.2f}`.

- `:` introduces the formatting instructions.
- `.` indicates that formatting information for the digits after the decimal point follows.
- `2` is the number of decimal places to display.
- `f` specifies the representation of a floating-point number 'float'.

```
print(f"The ratio of {number1} to {number2} is {ratio:.2f}.")
```

```
The ratio of 5 to 7 is 0.71.
```

The same is possible with a value:

```
print(f"The ratio is more precisely {0.7142857142857143:.3f}.")
```

```
The ratio is more precisely 0.714.
```

A value for the total number of digits to display before the decimal point can be specified `{ratio:7.2f}`, or including leading zeros `{ratio:07.2f}`:

```
print(f"The ratio of {number1} to {number2} is {ratio:7.2f}.")
print(f"The ratio with more precision is {0.7142857142857143:07.3f}.")
```

```
The ratio of 5 to 7 is    0.71.
The ratio with more precision is 000.714.
```

The decimal point counts as one character.

Commonly used format specifications are:

1.4 Integers

Integers use the output type `d`.

Format	Output
`nd`	Uses n characters for the output
`0nd`	Uses n characters, padding with zeros if necessary
`+d`	Always shows the sign, even for positive numbers

1.5 Floating-Point Numbers

Floating-point numbers use the output types `f` and `e`.

Format	Output
`.mf`	Uses m digits after the decimal point
`n.mf`	Uses n characters total, with m digits after the decimal point
`n.me`	Same as above, but in scientific notation

1.6 Strings

Strings use the output type `s`.

Format	Output
`ns`	Uses n characters total
`<ns`, `>ns`, `^ns`	Left-align, right-align, or center the string within n characters

A complete list of all available format types can be found in the Python documentation.

1.7 Exercises

1. Floating-point numbers can be displayed in standard form, e.g., {1015.39:12.4f}, or in scientific notation, {1015.39:e}.
   - Modify the standard form so that only one digit appears after the decimal point. What do you notice?
   - Modify the scientific notation so that the exponent is displayed with `E` instead of `e`.
2. Convert the number 1015.39 to a string and display it right-aligned in a field of 12 characters.
3. Using formatted strings, print a table showing the columns x, x^2, and x^3 for integers from -2 to 2.

Tipp 1: Sample Solution Output

1. Exercise

```
# Modify the standard form so that only one digit appears after the decimal point.
# What do you notice?
print(f"standard form: {1015.39:12.4f}")
print(f"scientific form: {1015.39:e}")

print(f"form with one decimal place: {1015.39:12.1f}")

# Modify the scientific notation so that the exponent uses 'E' instead of 'e'.
print(f"change exponent to 'E': {1015.39:E}")
```

```
standard form:    1015.3900
scientific form: 1.015390e+03
```

```
form with one decimal place:       1015.4
change exponent to 'E': 1.015390E+03
```

It is noticeable that the decimal place of the natural number is automatically rounded. When changing the decimal base to a large **E**, it will also be displayed as uppercase in the `print` output.

2. Exercise

```python
# 2.
# Convert the number 1015.39 into a string and display it right-aligned in 12 characters.
print(f"{"1015.39":>12s}")
```

```
     1015.39
```

3. Exercise

```python
# 3.
# Using formatted strings, print a table showing the columns x, x², and x³
# for integer values between -2 and 2.

x = -2
print(f"{x:>10d} {x**2:>10d} {x**3:>10d}")
x = -1
print(f"{x:>10d} {x**2:>10d} {x**3:>10d}")
x = 0
print(f"{x:>10d} {x**2:>10d} {x**3:>10d}")
x = 1
print(f"{x:>10d} {x**2:>10d} {x**3:>10d}")
x = 2
print(f"{x:>10d} {x**2:>10d} {x**3:>10d}")
```

```
        -2          4         -8
        -1          1         -1
         0          0          0
         1          1          1
         2          4          8
```

Alternatively, a (more elegant) solution using a `for` loop is also possible:

```python
for x in range(-2, 3):
    print(f"{x:>12d} {x**2:>12d} {x**3:>12d}")
```

```
-2          4          -8
-1          1          -1
 0          0           0
 1          1           1
 2          4           8
```

Sample solution by Marc Sönnecken.

(Arnold 2023a; Arnold 2023b; Arnold 2023c)

2 Data Types

Objects that store data have a data type. The data type specifies how the stored values should be interpreted by Python. For example, the value "1" in Python can represent a character, an integer, a Boolean value, the month of January, the weekday Tuesday, or the level of a categorical variable.

In this section, the data types that are most important for data analysis are introduced.

2.1 Numbers

Numbers include integers, Boolean values (truth values), floating-point numbers, and complex numbers (which are not discussed further here).

2.1.1 Integers

Integers are entered in decimal notation by default and can be positive or negative.

```
print(12, -8)
```

```
12 -8
```

In addition, integers can also be specified in other bases:

- Binary system: digits `0` and `1` with the prefix `0b`

```
print(0b1000, "plus", 0b0000, "plus", 0b0010, \
      "plus", 0b0001, "is", 0b1011)
```

```
8 plus 0 plus 2 plus 1 is 11
```

- Octal system: digits `0` to `7` with the prefix `0o`

```
print(0o7000, "plus", 0o0700, "plus", 0o0020, \
      "plus", 0o0000, "is", 0o7720)
```

```
3584 plus 448 plus 16 plus 0 is 4048
```

- Hexadecimal system: digits `0` to `F` with the prefix `0x`

```
print(0xF000, "plus", 0x0200, "plus", 0x00A0, \
      "plus", 0x0001, "is", 0xF2A1)
```

```
61440 plus 512 plus 160 plus 1 is 62113
```

2.1.2 Floating-Point Numbers

Floating-point numbers are written either with the decimal point `.` or in exponential notation. Floating-point numbers have the type `float`.

```
print(120.6, 1206e-1, 12060e-2, "\n")

print("When representing lottery winnings in exponential notation, the sign matters.")
print(1e-3, "or", 1e+3)
```

```
120.6 120.6 120.6

When representing lottery winnings in exponential notation, the sign matters.
0.001 or 1000.0
```

Since computers operate in the binary system, decimal numbers cannot be stored exactly. For example, the division of 1 by 10 in decimal is 0.1. In binary, 1 divided by 1010 is a repeating fraction:

$$\frac{1_2}{1010_2} = 0,000\overline{1100}_2$$

Therefore, decimal numbers must be approximated as a fraction of two integers (the binary fraction that approximates 0.1 is, in decimal notation, 3602879701896397 / 2^{55}). As a result, multiple floating-point numbers can be represented by the same binary approximation. Python outputs the shortest decimal representation, but since calculations are performed in binary, the nearest binary approximation—and therefore the corresponding shortest decimal—can change during computations (more information in the Python documentation).

```
print(0.1)  # The shortest decimal representation of the binary approximation
print(format(0.1, '.17g'))  # The next longer decimal representation of the same binary appro
print(0.3 - 0.2)  # Computed in binary, the binary approximation may change
```

```
0.1
0.10000000000000001
0.09999999999999998
```

In practical work with Python, you may occasionally encounter results that seem unusual at first glance.

```
print(0.1 + 0.2)
print(0.01 + 0.02)
print(0.001 + 0.002)
print(0.0001 + 0.0002)
print(0.00001 + 0.00002)
```

```
0.30000000000000004
0.03
0.003
0.00030000000000000003
3.0000000000000004e-05
```

2.2 Arithmetic Operators

Arithmetic operators allow you to perform basic calculations. The result is usually of type `float`, except when both operands are of type `int` and the result can be represented as an integer.

Operator	Description
`+`, `-`	Addition / Subtraction
`*`, `/`	Multiplication / Division
`//`, `%`	Integer Division / Remainder
`**`	Exponentiation

When combining multiple operators, their order must be respected or enforced using parentheses, e.g., `(1 + 2) * 3`. The same rules as in standard arithmetic apply. A complete overview of

operator precedence is available in the Python documentation. For arithmetic operators, the precedence from highest to lowest is:

Operator
`**`
`*`, `/`, `//`, `%`
`+`, `-`

Operators of the same precedence are evaluated from left to right.

2.3 Number Exercises

Solve the following exercises using Python.

1. 4 + 2 * 4 = ?
2. 2 to the power of 12 = ?
3. What is the remainder when 315 is divided by 4?
4. 1 + 26 / 5 = ?
5. What decimal number does 11111101001 represent?
6. 11111101001 / 101 = ?
7. Which investment yield is higher if 1000 euros are invested?
   a) 20 years with an annual return of 3%
   b) 30 years with an annual return of 2%

Tipp 2: Sample Solution Numbers

```python
# 1.
print('4 + 2 * 4 =', 4+2*4, '\n')
# 2.
print('2 to the power of 12 =', 2**12, '\n')
# 3.
print('Remainder of 315 divided by 4 =', 315%4, '\n')
# 4.
print('1 + 2^6 / 5 =', 1+(2**6)/5, '\n')
# 5.
print('binary 11111101001 is decimal =', 0b11111101001, '\n')
# 6.
print('binary 11111101001 / 101 is decimal =', 0b11111101001/0b101, '\n')
# 7.
option_a = 1000*1.03**20
print(f'after 20 years at 3% = {option_a:.2f}')
option_b = 1000*1.02**30
print(f'after 30 years at 2% = {option_b:.2f}')
```

```
4 + 2 * 4 = 12

2 to the power of 12 = 4096

Remainder of 315 divided by 4 = 3

1 + 2^6 / 5 = 13.8

binary 11111101001 is decimal = 2025

binary 11111101001 / 101 is decimal = 405.0

after 20 years at 3% = 1806.11
after 30 years at 2% = 1811.36
```

Sample Solution by Marc Sönnecken

2.4 Boolean Values

The boolean values `True` and `False` are the result of logical expressions, which we will explore in more detail later. They also take the values `1` and `0` and therefore belong to the number type in Python.

```
print("Is 10 greater than 9?", 10 > 9)
print("Is 11 less than 10?", 11 < 10)
print("Is 10 exactly 10.0?", 10 == 10.0, "\n")

print("True and False can be added using +:", True + False)
print("... and multiplied using *:", True * False, "\n")
```

```
Is 10 greater than 9? True
Is 11 less than 10? False
Is 10 exactly 10.0? True

True and False can be added using +: 1
... and multiplied using *: 0
```

Multiplying Boolean values is useful for combining multiple logical conditions into a logical AND:

```
print("Is 10 > 9 AND 10 > 8?", (10 > 9) * (10 > 8))
```

```
Is 10 > 9 AND 10 > 8? 1
```

The `bool()` function returns the truth value of a value.

```
print("Is 10 > 9 AND > 8?", bool((10 > 9) * (10 > 8)))
```

```
Is 10 > 9 AND > 8? True
```

Most values in Python have the truth value `True`.

```
print(bool(1), bool(2), bool(2.4))
print(bool('a'), bool('b'), bool('ab'))
```

```
True True True
True True True
```

Besides `False` and `0`, empty or non-existent values or objects evaluate to `False`.

```
print(bool(False), bool(0))
print(bool("")) # an empty string
print(bool([])) # an empty list
print(bool(())) # an empty tuple
print(bool({})) # an empty dictionary
print(bool(None)) # None represents a non-existent value
```

```
False False
False
False
False
False
False
```

The collection types list, tuple, and dictionary will be introduced in the following chapters. Boolean values can control the execution of program code by acting like `on` and `off`. This way, code with an `if` statement is only executed if a collection type actually contains values.

```
my_list = ['Apples', 'Butter']
if my_list:
    print(f"We need to buy {my_list}.")

my_list = []  # an empty list
if my_list:
    print(f"We need to buy {my_list}.")
```

```
We need to buy ['Apples', 'Butter'].
```

2.5 Logical Operators

Logical operators include the logical connections `and`, `or`, and `not`. In addition, comparison operators such as `>`, `>=`, or `==` can also be used. The result of these operations is of type `bool`. Operators are executed in the following order. Operators of the same precedence are executed from left to right.

Operator	Description
`&`	bitwise AND
`^`	bitwise XOR
`|`	bitwise OR
`<`, `<=`, `>`, `>=`, `!=`, `==`	less than / less than or equal / greater than / greater than or equal / not equal / equal
`not`	logical NOT
`and`	logical AND
`or`	logical OR

Warning 1: Bitwise Operators

Special caution is required with bitwise operators. These do not compare numbers as a whole, but rather bit by bit (in the binary system). Note that bitwise operators have higher precedence than comparison operations.

```
print(10 > 5 and 10 > 6)
print(10 > 5 & 10 > 6)
print(5 & 10)
print(10 > False > 6)
print((10 > 5) & (10 > 6))
```

```
True
False
0
False
True
```

Tip

In general, bitwise operators are not needed for data analysis. Avoid unnecessary mistakes: do not use the bitwise operators `&`, `^`, and `|`.

However, the operators `&`, `^`, and `|` have a different meaning for sets (which we will cover later). In other modules, these operators also have a different syntactic role, e.g., in the Pandas package when passing multiple slicing conditions: `df = df[(condition1) & (condition2) | (condition3)]`.

2.6 Boolean Value Exercises

Solve the exercises using Python.

1. Is the ratio of 44 to 4.5 greater than 10?
2. Is it true that 4.5 is greater than 4 but less than 5?
3. Is 2 to the power of 10 equal to 1024?
4. Are the numbers 3, 4, and 5 divisible by 2 OR not equal to 10?
5. Check whether a person has to pay the full price based on their age.
   Adults pay full price. Children under 14 travel for free. Teenagers from 14 up to 18 years old and seniors from 65 years onwards get a discount.

Tipp 3: Sample Solution for Boolean Values

```
# 1.
print('Is 44/4.5 greater than 10?', 44/4.5 > 10)
# 2.
print('Is 4.5 greater than 4 but less than 5?', 4 < 4.5 and 4.5 < 5)
# 3.
print('Is 2 to the power of 10 equal to 1024?', 2**10 == 1024)
# 4.
print('Are the numbers 3, 4, and 5 divisible by 2 OR not equal to 10?')
print('3:', 3%2 == 0 or 3 != 10)
print('4:', 4%2 == 0 or 4 != 10)
print('5:', 5%2 == 0 or 5 != 10)
# 5. Check age

# Variable for the age to check
age = 17
print('Age =', age)

print('The person has to pay full price:', age >= 18 and age < 65)
print('The person gets a discount:', age >= 14 and age < 18 or age >= 65)
print('The person travels for free:', age < 14)
```

```
Is 44/4.5 greater than 10? False
Is 4.5 greater than 4 but less than 5? True
Is 2 to the power of 10 equal to 1024? True
Are the numbers 3, 4, and 5 divisible by 2 OR not equal to 10?
3: True
4: True
5: True
Age = 17
The person has to pay full price: False
The person gets a discount: True
The person travels for free: False
```

Sample Solution by Marc Sönnecken and Maik Poetzsch

2.7 Strings

Strings are enclosed in single or double quotes in Python.

```
print('a string')
print("another string")
```

```
a string
another string
```

Within a string, single or double quotes can be used, as long as they don't match the quotes surrounding the string.

```
print('A sophisticated heap beam, which we call a "LASER".')
print("I've turned the moon into what I like to call a 'Death Star'.")
```

```
A sophisticated heap beam, which we call a "LASER".
I've turned the moon into what I like to call a 'Death Star'.
```

The control character \ (also called the escape character) allows you to use certain special characters.

```
print("The control character \\ allows the same \"quotation marks\" in the output of print.")
print("First a\tTab, then a\nnew line.")
```

```
The control character \ allows the same "quotation marks" in the output of print.
First a Tab, then a
new line.
```

A prefixed `r` makes the backslash \ treated literally (raw string literal). This is useful, for example, when working with file paths.

```
print("The data is located at: C:\tolle_daten\nordpol\weihnachtsmann")
print(r"The data is located at: C:\tolle_daten\nordpol\weihnachtsmann")
```

```
The data is located at: C:  olle_daten
ordpol\weihnachtsmann
The data is located at: C:\tolle_daten\nordpol\weihnachtsmann
```

2.8 String Operations

Some operators also work with data of type string.

```
# string + string
print('a' + 'b')

# string * number
print(15 * 'a')

# logical operators
print('a' < 'b', 'a' >= 'b', 'a' != 'b')
print('a' or 'b', 'a' and 'b')
```

```
ab
aaaaaaaaaaaaaaa
True False True
a b
```

2.9 String Exercises

Solve the exercises using Python.

1. What happens if you add the string `"Python"` with `"for beginners"`?
2. Create a string that contains the string `"tick tack"` 10 times.
3. Which string is smaller, `"Aachen"` or `"Bern"`? Why is that, and how are strings compared?
4. Print the file path: `"~\home\tobi\new_data"`

Tipp 4: Example Solution for String Exercises

```
# 1.
print("Python" + " for beginners")  # The two strings are concatenated directly, without ar
# 2.
print(10 * "tick tack")
# 3.
print("Aachen" < "Bern")  # Unicode values of the characters are compared
# 4.
print(r"~\home\tobi\new_data")  # Python interprets the backslash "\" as an escape characte
```

```
Python for beginners
tick tacktick tacktick tacktick tacktick tacktick tacktick tacktick tacktick tacktick tack
```

```
True
~\home\tobi\new_data
```

Sample Solution by Marc Sönnecken

2.10 Variables

Variables are placeholders or references to data. Assignment is done using the assignment operator `=`. A variable name may only consist of letters, numbers, and underscores. The first character cannot be a number.

```
var_1 = 'ABC'
var_2 = 26
var_3 = True

print("The", var_1, "has", var_2, "letters. This is", var_3)
```

```
The ABC has 26 letters. This is True
```

In Python, variables do not need to be initialized. The data type of a variable is determined by assigning an appropriate value.

```
print("The variable var_1 has the type", type(var_1))
print("The variable var_2 has the type", type(var_2))
print("The variable var_3 has the type", type(var_3))
```

```
The variable var_1 has the type <class 'str'>
The variable var_2 has the type <class 'int'>
The variable var_3 has the type <class 'bool'>
```

The data type of a variable changes when it is assigned a new value of a different data type.

```
var_1 = 100
print("The variable var_1 has the type", type(var_1))
```

```
The variable var_1 has the type <class 'int'>
```

Similarly, the data type of a variable can change to accommodate the result of an operation. In this example, the result cannot be stored as an integer. Python therefore assigns the data type 'float' to the object `var_1`.

```
var_1 = var_1 / 11
print("The variable var_1 has the type", type(var_1))
```

```
The variable var_1 has the type <class 'float'>
```

Python provides functions to determine the data type of a variable and to convert it. The function `type()` has already been used in the code examples to check the data type (i.e., class) of objects. The following code block demonstrates type conversion.

```
a = 67
print(a, type(a))

b = a + 1.8
print(b, type(b), "\n")

print(f"Note the truncation of the decimal part:\nConverting to integers with int(): {(a := i

print(f"Converting to ASCII characters with chr(): {(a := chr(a)), (b := chr(b))}\n")

print(f"Converting to an ASCII number with ord(): {(a := ord(a)), (b := ord(b))}\n")

print(f"Converting to floating-point numbers with float(): {(a := float(a)), (b := float(b))

print(f"Converting to strings with str(): {(a := str(a)), (b := str(b))}\n")

print(f"Converting to Boolean values with bool(): {bool(a), bool(b)}")
```

```
67 <class 'int'>
68.8 <class 'float'>

Note the truncation of the decimal part:
Converting to integers with int(): (67, 68)

Converting to ASCII characters with chr(): ('C', 'D')

Converting to an ASCII number with ord(): (67, 68)
```

```
Converting to floating-point numbers with float(): (67.0, 68.0)

Converting to strings with str(): ('67.0', '68.0')

Converting to Boolean values with bool(): (True, True)
```

2.10.1 Additional Assignment Operators

In the code block above, the so-called Walrus Operator `:=` was used. This allows assignments to be made within an expression (here, inside the `print()` function). Python has a variety of other assignment operators (see the Python documentation or a more user-friendly overview here).

Operator	Equivalent Assignment
a += 2	a = a + 2
a -= 2	a = a - 2
a *= 2	a = a * 2
a /= 2	a = a / 2
a %= 2	a = a % 2
a //= 2	a = a // 2
a **= 2	a = a ** 2

Tipp 5: Readable Assignments

The assignment operators shown in the table are easy to read and understand for someone reviewing your code, as assignments always appear at the start of a line, aligned to the left.
In contrast, the Walrus Operator can appear anywhere in your code. While this can be convenient while writing, it can reduce readability. If you use the Walrus Operator, make sure your code remains understandable.

2.10.2 Naming Variables

There are usually only a few rules for naming variables. Still, it is better to choose a long but descriptive variable name than a short one that is easy to type. Code is read far more often than it is written. Can you remember? What value is stored in the variable `Var_3`, and what values are stored in `variable1` and `a`? It's enough if you can recall the correct data type.

If you can't remember, this is an example of poor naming: these variable names are not descriptive. The solution is in the expandable section below.

Tipp 6: Variable Solution

```
print(var_3, type(var_3))
print(variable1, type(variable1))
print(a, type(a))
```

```
True <class 'bool'>
15 <class 'int'>
67.0 <class 'str'>
```

Therefore, it is recommended to use "descriptive," i.e., self-explanatory, variable names. Self-explanatory variable names are those where the name describes the content of the variable. For example, if a variable stores a student's degree, it can be named `academic_degree` or `studienabschluss`. If data from different years are being processed, the year can be included in the variable name for better distinction, e.g., `academic_degree_2023` or `studienabschluss2024`. This improves code readability and makes using the variable easier. More information can be found in this Wikipedia section.

Warnung 2: Keywords and Function Names

Reserved keywords and function names in Python are unsuitable as variable names. While Python will raise a syntax error if you try to assign a value to keywords like `True` or `break`, function names can be reassigned, for example with `print = 6`. After doing so, calling the `print` function will no longer work. In this case, you must remove the assignment from the script and restart Python.
The following keywords exist in Python:

```
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
await      else       import     pass       True
break      except     in         raise      class
False      finally    is         return     None
```

2.11 Variable Exercises

1. Write a script that converts a given time in seconds into the number of days, hours, minutes, and seconds, and prints this breakdown.
   Calculate the breakdown for the following times:

a) 79,222 s

b) 90,061 s

c) 300,000 s

2. The position of a vehicle at time t, which accelerates constantly with acceleration a, is given by:

$$x(t) = x_0 + v_0 \cdot t + \frac{1}{2} \cdot at^2$$

Here, x_0 is the initial position and v_0 is the initial velocity. Create variables for x_0, v_0, and a, and assign them values. Since the variables only store numerical values without units, consider any necessary unit conversions. You can use the following example values:

$$x_0 = 10\,km \;\; v_0 = 50\frac{km}{h} \;\; a = 0.1\frac{m}{s^2}$$

Create a variable for the time t, e.g.: $t = 10\,min$.
Use the above equation and your variables to calculate the position $x(t)$.
Output not only the value of $x(t)$ in kilometers, but embed it in a full sentence including units.

Tipp 7: Sample Solution for Variable Exercises

1. Exercise

```python
# 1.
def time_breakdown(t):
    days = t // (24*3600)
    t = t % (24*3600)
    hours = t // 3600
    t = t % 3600
    minutes = t // 60
    seconds = t % 60

    return days, hours, minutes, seconds

# Enter the values to check: a, b, c
given_time = 79222

days, hours, minutes, seconds = time_breakdown(given_time)

print(f"{given_time} seconds correspond to {days} days, {hours} hours, {minutes} minutes, a
```

```
79222 seconds correspond to 0 days, 22 hours, 0 minutes, and 22 seconds
```

2. Exercise

```python
# 2.
# Initial values including conversions
x0 = 10*1000 # kilometers to meters
v0 = 50/3.6 # km/h to m/s
a = 0.1
t = 10*60 # minutes to seconds

x = x0 + v0 * t + 0.5 * a * t **2

print(f"The calculated position of the vehicle at time t = {(t/60):.0f} minutes is {(x/1000
```

```
The calculated position of the vehicle at time t = 10 minutes is 36.33 kilometers.
```

Sample Solution by Marc Sönnecken

(Arnold 2023a; Arnold 2023b)

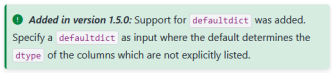
3 Functions: Basics

Functions are subroutines that bundle program instructions so that parts of a program can be reused multiple times. This allows a program to be written more quickly and makes it easier to read. As you can see in the documentation, Python comes with a manageable number of basic functions. This chapter teaches the general usage of functions included in Python.

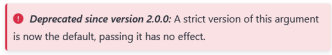
Python is continuously evolving: new versions with new features are released regularly. Therefore, this chapter also provides a number of tips on how to read Python's documentation. This is especially relevant when extending Python extensively with modules. For example, functions in the Pandas module often have dozens of documented parameters.

Tipp 8: Documentation

The most important tip first: **Use the documentation!** Even if you are familiar with a function, make sure to check that you are up to date. This way, you get a complete overview of default and optionally available parameters. You also notice changes in function behavior and avoid unexpected errors.



(a) New in Python



(a) Deprecated in Python

Pay attention to the correct version of the documentation.

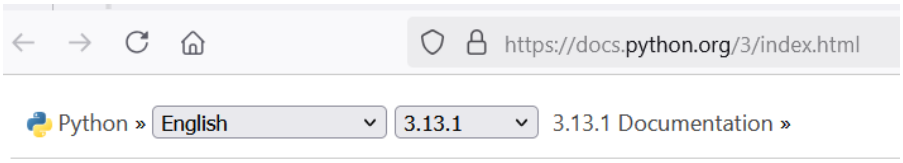


Abbildung 3.3: Documentation version selector

3.1 Functions and Methods

In Python, there are two types of callable objects: functions and methods.

3.1.1 Functions

Functions can take objects of any data type as input.
Functions are called using their function name followed by parentheses `()`. An example is the `print()` function:

```
var_str = 'ABC'
var_int = 26
var_bool = True

print("The variable var_str has type", type(var_str))
print("The variable var_int has type", type(var_int))
print("The variable var_bool has type", type(var_bool))
```

```
The variable var_str has type <class 'str'>
The variable var_int has type <class 'int'>
The variable var_bool has type <class 'bool'>
```

Functions must always return a value. If functions cannot or should not return a value, the value `None` is returned, which indicates a non-existent value.

```
res = print(15)
print(res)
```

```
15
None
```

Functions can be nested and thus executed from the inside out in sequence. In this code example, first the sum of two numbers is calculated, and then the boolean value of the result is determined. This value is then output using the `print` function.

```
print(bool(sum([1, 2])))
```

```
True
```

3.1.2 Methods

Methods are a feature of object-oriented programming languages. In the previous chapter, it was explained that in Python, objects belong to a specific type or class, and depending on the values stored within them, they have an appropriate data type. Methods are functions that belong to a specific class and are only available for objects of that class. Methods can also be defined for multiple classes. Methods are called by appending a dot `.` to the object followed by the method name: `variable.method` or `(value).method`. For example, `.upper()`, `.lower()`, and `.title()` are methods defined for strings.

```
awesome_text = "Python 3.12 is awesome."

print(awesome_text.upper())
print(awesome_text.lower())
print(awesome_text.title(), "\n")

print(("It also works with values placed in parentheses.").upper())
```

```
PYTHON 3.12 IS AWESOME.
python 3.12 is awesome.
Python 3.12 Is Awesome.

IT ALSO WORKS WITH VALUES PLACED IN PARENTHESES.
```

Methods like `.lower()` are not available for objects with an inappropriate data type.

```
print((1).upper())
```

```
'int' object has no attribute 'upper'
```

Methods can be chained and executed one after another. In this example, the string 'Cat' is converted to lowercase, and then the frequency of the letter 'c' is counted.

```
print('Cat'.lower().count('c'))
```

```
1
```

Which methods are available for an object can be determined using the function `dir(object)`. However, the output of this function is often very extensive. To select the relevant entries, the output needs to be filtered. This is not strictly necessary—interested users can refer to Beispiel 1.

Hinweis 1: Determining an Object's Methods

The function `dir(object)` can be used to display the methods available for an object. However, it also shows the attributes and methods of the object's class, so the output is often very extensive. For example, for the integer 1:

```
print(dir(1))
```

```
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__di
```

To limit the output to methods, the following list comprehension can be used:

```
object_ = 1

methods = [attr for attr in dir(object_) if callable(getattr(object_, attr))]
print(methods)
```

```
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__di
```

Methods enclosed with double underscores are defined for the class. The following function removes methods with double underscores from the output:

```
object_instance = 1

attributes = [attr for attr in dir(object_instance) if (callable(getattr(object_instance, a
print(attributes)
```

```
['as_integer_ratio', 'bit_count', 'bit_length', 'conjugate', 'from_bytes', 'is_integer', 't
```

In the case of an integer, methods (to distinguish from floating-point numbers in enclosing parentheses) can be called as follows:

```
(1).as_integer_ratio()
```

```
(1, 1)
```

Methods of the object 'awesome_text':

```
object = awesome_text

attributes = [attr for attr in dir(object) if (callable(getattr(object, attr)) and not attr
print(attributes)
```

```
['capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find',
```

3.2 Parameters

Many functions and methods can accept multiple parameters, separated by commas. The values passed to a function are called arguments (Python Documentation). Parameters control the execution of a program. The parameters available for the `print()` function are listed in the function documentation:

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

`*objects`, `sep`, `end`, `file`, and `flush` are the parameters of the `print()` function.

- Parameters without an equal sign `=` must be provided when calling the function or method. Parameters with an equal sign `=` are optional and can be provided during the call.
- The values after the equal sign indicate the default values of the parameters. These are used if an argument is not explicitly passed during the call.

Tipp 9: Exceptions for Default Values

The values listed in a function's definition are not always the actual default values. Therefore, it is recommended to read the parameter descriptions when using a function. Some functions use the keyword `None` to indicate the default value. The value `None` serves as a placeholder. An example is the NumPy function numpy.loadtxt().

```
numpy.loadtxt(fname, dtype=<class 'float'>, comments='#', delimiter=None, converters=None,
              skiprows=0, usecols=None, unpack=False, ndmin=0, encoding=None, max_rows=None
              *, quotechar=None, like=None)
```

- For the parameter `delimiter`, the default value is listed as the keyword `None`. However, according to the function description, the actual default is whitespace: "The default is whitespace."
- The parameter `usecols` also has the default value `None`: "The default, None, results in all columns being read."

Another example is the function pandas.read_csv(). Some arguments have the default value `<no_default>`. (Only selected parameters are shown below).

```
pandas.read_csv(sep=<no_default>, verbose=<no_default>)
```

From the description, the actual default values can be seen:
sep : str, default ','
verbose : bool, default False

- In Python, arguments can either be passed as a positional argument. This means Python expects arguments in a fixed order corresponding to the parameters in the function definition. Alternatively, arguments can be passed as keyword arguments, where inputs are mapped according to the parameter name. By default, arguments can be passed either positionally or by keyword. Deviations from this are indicated by the symbols `*` and `/` (see the tip below).

Tipp 10: Positional and Keyword Arguments, *args and **kwargs

The symbols `*` and `/` indicate which parameters can or must be passed positionally or as keyword arguments.

Left side	Separator	Right side
positional-only arguments	/	positional or keyword arguments
positional or keyword arguments	*	keyword-only arguments

(https://realpython.com/python-asterisk-and-slash-special-parameters/)

An example of the `*` separator is the `glob` function from the module of the same name. The `pathname` parameter can be passed positionally (first position) or as a keyword. The other parameters must be passed as keyword arguments.

```
glob.glob(pathname, *, root_dir=None, dir_fd=None, recursive=False, include_hidden=False)
```

Both control characters can appear within a function definition, but only in the order `/` and `*`. In the reverse order, it would be impossible to pass arguments. An example is the `sorted` function. The first parameter must be passed positionally, while the `key` and `reverse` parameters must be passed as keywords.

```
sorted(iterable, /, *, key=None, reverse=False)¶
```

> **Exceptions**
>
> Some functions deviate from the usual pattern, for example the functions `min()` and `max()`. They are defined (among others) in the following form:
>
> ```
> min(iterable, *, key=None)
> max(iterable, *, key=None)
> ```
>
> Both functions accept the `iterable` parameter, but not as a keyword.

Many functions can accept an arbitrary number of arguments, either positionally or as keywords. This is generally indicated by the keywords `*args` (positional arguments) and `**kwargs` (keyword arguments). The difference is marked by the single or double asterisk, and the keywords themselves can be replaced (as with the function `print(*objects)`). The keyword `*args` also corresponds to the `*` symbol in the function definition, meaning that only keyword arguments can follow it. More information can be found here.

In the function definition of `print()`, `*objects` is a positional parameter (always in the first position), has no default value, and can accept an arbitrary number of arguments (n inputs are placed in the first n positions). The other parameters of the `print()` function are optional and must be passed as keyword arguments.

3.3 Function Exercises

1. True or False: Methods are available depending on the data type of a value or an object.

2. Print the three values 1, 2, and 3 using `print()`. Parameterize the function so that the output looks like this:

```
1_x_2_x_3
```

3. Look up the function bool() in the documentation.

- Which parameters does the function accept, and which of them are optional?
- Which arguments are passed positionally and which can be passed as keyword arguments? Is the method of passing fixed or flexible?

Solutions

Exercise 1: correct
Exercise 2

```
print(1, 2, 3, sep="_x_")
```

Task 3: The function bool() has an optional argument object with a default value of False. The argument must be passed positionally.

4 Control Flow

Control flow allows you to manage the execution of parts of a program. Statements can be skipped or executed multiple times.

4.0.1 Branching

Branching allows for conditional execution, where different parts of the script run depending on one or more conditions.

In Python, branching starts with the keyword `if`. This is followed by the branch condition and ends with a colon `:`. If the branch condition is true, the indented block of statements is executed.

```
if condition:
    block_of_statements
```

```python
# Example: Number smaller than a threshold

a = 7
if a < 10:
    print('The number', a, 'is smaller than 10.')
```

```
The number 7 is smaller than 10.
```

It is also possible to define an alternative block of statements that will be executed if the condition is false. This is implemented using the `else` keyword.

```
if condition:
    # condition is true
    statement_block
else:
    # condition is false
    statement_block
```

```python
# Example: Number below a threshold with alternative output

a = 13
if a < 10:
    print('The number', a, 'is less than 10.')
else:
    print('The number', a, 'is not less than 10.')
```

```
The number 13 is not less than 10.
```

Multiple conditions can also be used.

```python
# Example: Number within the range between 10 and 20

num = 1
if num < 20 and num > 10:
    print('The number', num, 'is between 10 and 20.')
else:
    print('The number', num, 'is not between 10 and 20.')
```

```
The number 1 is not between 10 and 20.
```

Finally, several alternative conditions can be checked. This is possible by nesting branches.

```python
# Example: Number within the range between 10 and 20 using nested conditions

number = 12
if number > 10:
    print('The number', number, 'is greater than 10.')

    if number < 20:
        print('The number', number, 'is less than 20.')
        print('Therefore, the number is between 10 and 20.')
    else:
        print('The number', number, 'is greater than 20 and not within the desired range.')
else:
    print('The number', number, 'is less than 10 and not within the desired range.')
```

```
The number 12 is greater than 10.
The number 12 is less than 20.
Therefore, the number is between 10 and 20.
```

On the other hand, this is possible with the keyword `elif`.

```
# Example: Number in the range between 10 and 20 using elif

number = 112
if number < 20 and number > 10:
    print('The number', number, 'is between 10 and 20.')
elif number < 10:
    print('The number', number, 'is less than 10 and not in the desired range.')
elif number > 20 and number <= 100:
    print('The number', number, 'is greater than 20, but not greater than 100.')
elif number > 20 and number <= 1000:
    print('The number', number, 'is greater than 20, but not greater than 1000.')
else:
    print('The number', number, 'is not between 10 and 20 and is greater than 1000.')
```

```
The number 112 is greater than 20, but not greater than 1000.
```

4.0.2 Loops

Loops allow you to repeat instructions. In Python, `while` and `for` loops can be defined. Both require:

- a **loop header**, which controls the execution of the block of statements, and
- a **block of statements**, i.e., a group of instructions that is executed in each iteration of the loop.

The `while` loop requires only a single condition in the loop header and is the more general of the two. Any `for` loop can be rewritten as a `while` loop (by integrating a counter into the block of statements). Which type is used depends on the specific task.

while Loops

A `while` loop repeatedly executes the block of statements as long as the execution condition is true. The loop starts with the keyword `while`, followed by the execution condition. The loop header ends with a colon `:`. The indented block of statements is defined below it.

```
while condition:
    statement block
```

At the start of the loop and after each iteration, the condition is checked. If it is true, the block of statements is executed; if not, the loop ends and the next statement outside the loop is executed.

```
# Example: Incrementing a variable value

# Set starting value
a = 5

# Define a loop that runs as long as a is less than or equal to 10
while a <= 10:
    # Loop block:

    # 1. Print the current value of a
    print('current value of a', a)

    # 2. Increment a by one
    a += 1

# Print the value after the loop
print('value of a after the loop', a)
```

```
current value of a 5
current value of a 6
current value of a 7
current value of a 8
current value of a 9
current value of a 10
value of a after the loop 11
```

Warning 3: Infinite Loop

`while` loops result in an infinite loop if the termination condition can never be met. For example, in the following loop, there is no way for the loop variable `x` to reach the value 5.

```
x = 1

while x < 5:
  print(x)
```

In this case, you can stop the program execution by pressing `Ctrl` + `C`.

for Loops

While the `while` loop runs as long as a condition is true, the `for` loop is controlled by a loop variable that iterates over a sequence. The syntax looks like this:

```
for loop variable in sequence:
  statement block
```

To define the loop header, the two keywords `for` and `in` are used, and the header ends with a colon `:`. The indented block following the header is the body of the loop.

The sequence is created using a range object, which is generated with the function `range(start=0, stop, step=1)`. `range()` accepts integer values as *positional arguments* and generates integers from `start` up to *but not including* `stop` with the step size `step`. It is important to note that Python counts **exclusively**, meaning it starts counting at 0 by default and the value passed as the `stop` argument is not included.

The `range()` function returns a range object, which does not immediately produce the expected output when used with `print()`.

```
# range(start=1, stop=5) - step is not provided, so the default value step=1 is used
print(range(1, 5), type(range(1, 5)))
```

```
range(1, 5) <class 'range'>
```

This behavior is called lazy evaluation (lazy evaluation): The values of the `range` object are only generated when they are needed. In the following code, the `range` object is iterated with a loop, and in each iteration, the value of the loop variable `i` is printed.

```python
for i in range(1, 5):
    print(i)
```

```
1
2
3
4
```

The `step` parameter allows you to control the increment.

```python
for i in range(1, 15, 3):
    print(i)
```

```
1
4
7
10
13
```

It is useful to convert the range object into a list or a tuple, collection types that will be covered in the next chapter.

```python
# Output even numbers from 1-10 as a list
print("List:", list(range(2, 11, 2)))

# Output odd numbers from 1-10 as a tuple
print("Tuple:", tuple(range(1, 11, 2)))
```

```
List: [2, 4, 6, 8, 10]
Tuple: (1, 3, 5, 7, 9)
```

`start` and `stop` can also be negative. For ascending sequences, `stop` must always be greater than `start`.

```python
for i in range(-5, -1):
    print(i)
```

```
-5
-4
-3
-2
```

`step` can also be negative. In that case, a descending sequence of numbers is generated. For descending sequences, `start` must always be greater than `stop`.

```python
for i in range(-1, -5, -1):
    print(i)
```

```
-1
-2
-3
-4
```

A descending sequence can also be created using the `reversed(sequence)` function. The result differs depending on the approach, or a different input is needed to get the same result:

```python
# Descending sequence, but start is greater than stop
# The output remains empty
print(list(range(5, 0)))

# Using the reversed function works
print(list(reversed(range(0, 5))))

# For a negative step size, start must be greater than stop
print(list(range(0, 5, -1)))
print(list(range(5, 0, -1)))
```

```
[]
[4, 3, 2, 1, 0]
[]
[5, 4, 3, 2, 1]
```

List Notation

The so-called list notation is a shorthand for for-loops. Loops written in list notation must fit on a single line and have the following syntax:

```
squared_numbers = [value ** 2 for value in range(10, 0, -1)]
print(squared_numbers)
```

```
[100, 81, 64, 49, 36, 25, 16, 9, 4, 1]
```

(Matthes (2017), p. 71)

The Keywords break and continue

Sometimes it may be necessary to exit the block of a loop prematurely. For this, the keywords `break` and `continue` can be used. The keyword `break` causes the loop to terminate immediately. In contrast, the keyword `continue` ends the current iteration and moves on to the next iteration of the loop.

```
x = 0
while x < 10:

  x += 1

  # skip even numbers
  if x % 2 == 0:
    continue

  # stop the loop when x == 7
  if x == 7:
    break

  print(x)
```

```
1
3
5
```

4.0.3 Exception Handling

Exception handling allows Python to execute alternative instructions when an error occurs. This is useful, for example, when reading datasets, as it lets you display the cause of error messages—a technique introduced in the Reading Structured Datasets module.

In Python, there are two types of errors. The first are syntax errors, which Python reports with an error message similar to the following. Syntax errors are resolved by writing syntactically correct code.

```
print(1}
```

```
closing parenthesis '}' does not match opening parenthesis
```

The second type of errors are exceptions, which can occur even in syntactically correct program code. Exceptions also result in error messages.

```
# Example 1: Division by zero
print(1 / 0)
```

```
division by zero
```

```
# Example 2: undefined variable
print(undefined_variable)
```

```
name 'undefined_variable' is not defined
```

Error messages like these can be handled in Python using exception handling. This is initiated with the keyword `try`, which is completed with a colon `:`. In the next line, an indented block of statements follows that should be tested for exceptions. *Note: The block of statements is actually executed, so changes to data or files are possible.* Afterwards, the keyword `except` is used—again followed by a colon `:` and, in the next line, an indented block of statements—to define what should happen when an exception occurs. Optionally, the keyword `else` can be used following the same pattern to define another block of statements that is executed *in addition* when the block under `try` runs successfully. The general structure looks as follows:

```
try:
    statement_block_1
except:
    statement_block_if_statement_block_1_raises_an_exception
else:
    optional_statement_block_if_statement_block_1_does_not_raise_an_exception
```

Using exception handling, we can display the elements that cause an error.

```python
a = 1
b = 2

try:
    difference = a - b
except:
    print(f"The difference between {a} and {b} could not be calculated.")
else:
    print(f"The difference between {a} and {b} is {difference}.")
```

```
The difference between 1 and 2 is -1.
```

```python
a = 1
b = 'abc'

try:
    difference = a - b
except:
    print(f"The difference between {a} and {b} could not be calculated.")
else:
    print(f"The difference between {a} and {b} is {difference}.")
```

```
The difference between 1 and abc could not be calculated.
```

It is also possible to intercept and output the error message. To do this, modify the line `except:` as follows: `except Exception as error:`

```python
a = 1
b = 'abc'

try:
    difference = a - b
except Exception as error:
    print(f"The difference between {a} and {b} could not be calculated.")
    print(error)
else:
    print(f"The difference between {a} and {b} is {difference}.")
```

```
The difference between 1 and abc could not be calculated.
unsupported operand type(s) for -: 'int' and 'str'
```

4.1 Control Flow Exercises

1. Write a program that counts from 1 to 25 and from 38 to 50, and prints every value that is divisible by 7 using `print()`.
2. Roulette: Write a program that checks whether a given number is within the valid range of the roulette table. If not, it should display an error message. If it is, the program should output whether the number is:

- even or odd,
- red, black, or green,
- low (1-18) or high (19-36), and
- in the 1st, 2nd, or 3rd dozen.

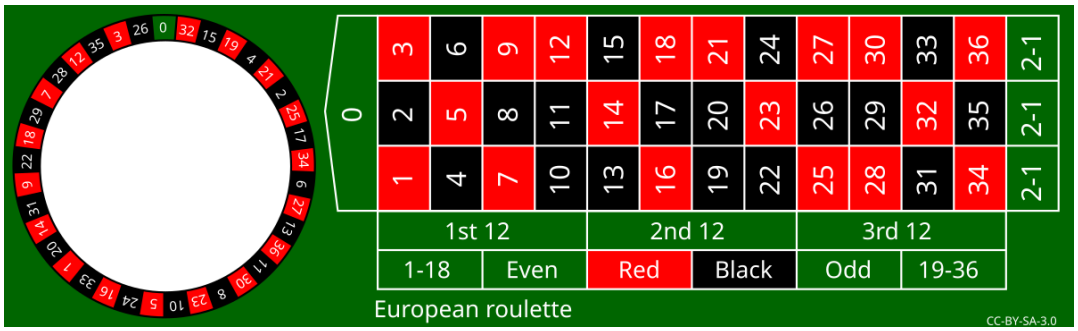


Abbildung 4.1: Roulette Table

European roulette by Betzaar.com is licensed under CC 3.0 BY-SA and available on wikimedia.org. 2010

Sample Solution for Control Flow Exercises

1. Exercise

```
# Write a program that counts from 1 to 25 and from 38 to 50, printing each value divisible

i = 0
while i < 50:
    i += 1
    if i > 25 and i < 38 or i % 7 != 0:
        continue
    else:
        print(i)

# Alternative using lists
# Note: as mentioned in the "Loops" section, the stop value of the range function must be c
numbers = list(range(1,26)) + list(range(38,51))
for i in numbers:
    if i % 7 == 0:
        print(i)
```

```
7
14
21
42
49
7
14
21
42
49
```

2. Exercise

```python
# Define the integer to check as a variable
x = 10

# The red and black numbers are defined as lists (a tuple would also work here)
red_numbers = [1,3,5,7,9,12,14,16,18,21,23,25,27,28,30,32,34,36]
black_numbers = [2,4,6,8,10,11,13,15,17,19,20,22,24,26,29,31,33,35]

# The first if-statement checks whether the number is within the valid range. If not, it ju
if x >= 0 and x <= 36:

    # Inside this if-statement, there are further if-statements that check other characteri

    # Check whether the number is even or odd
    if x % 2 == 0:
        print("Number is even")
    else:
        print("Number is odd")

    # Check whether the number is red, black, or green
    if x in red_numbers:
        print("Number is red")
    elif x in black_numbers:
        print("Number is black")
    elif x == 0:
        print("Number is green")

    # Check whether the number is high or low
    if x >= 1 and x <= 18:
        print("Number is low")
    elif x >= 19 and x <= 36:
        print("Number is high")

    # Check which dozen the number belongs to
    if x >= 1 and x <= 12:
        print("Number is in the 1st dozen")
    elif x >= 13 and x <= 24:
        print("Number is in the 2nd dozen")
    elif x >= 25 and x <= 36:
        print("Number is in the 3rd dozen")
    else:
        print("Number is not in any dozen")

else:
    print("Number is not within the valid range.")
```

```
Number is even
Number is black
Number is low
Number is in the 1st dozen
```

Sample solution by Marc Sönnecken and Maik Poetzsch

(Arnold (2023d))

5 Collection Types

Collection types are used to store and manage multiple values in a single variable. In Python, there are four collection types, each represented by its own class:

- Lists can contain a flexible number of elements of any type.
- Tuples can also contain elements of any type, but they are immutable.
- Sets are unordered collections that can contain each element only once.
- Associative Arrays, or dictionaries, are key-value mappings.

In this chapter, the four collection types are briefly introduced first. Then, we will focus on how to work with lists in particular.

5.1 Lists

Like all types in Python, lists are created by assignment. When defining a list, elements are enclosed in square brackets `[]` and separated by commas `,`. Lists can be concatenated with the `+` operator. `*` repeats a list n times.

```
text_variable = 'abc'

list1 = [1, 'xy', True, text_variable]
print(list1)

# Lists can also contain other lists
list2 = [None, list1]
print(list2)

# Lists can be concatenated with + and repeated with *
print(list1 + list2)
print(list1 * 2)
```

```
[1, 'xy', True, 'abc']
[None, [1, 'xy', True, 'abc']]
[1, 'xy', True, 'abc', None, [1, 'xy', True, 'abc']]
[1, 'xy', True, 'abc', 1, 'xy', True, 'abc']
```

An empty list can be created by assigning `[]`.

```
empty_list = []
print(empty_list)
```

```
[]
```

5.1.1 Slicing: the Access Operator []

Accessing individual or multiple elements of a list (and other collection types) is done using the access operator `[]`. A portion of an object is called a slice, which is why the operator is also called the slice operator.

Accessing Individual Elements

Elements are accessed by their index, starting at 0.

```
print(list1)
print(list1[0])
print(list1[3])
```

```
[1, 'xy', True, 'abc']
1
abc
```

Nested lists can be accessed using two consecutive indexing operators. The list `list2` contains, at index position 1, a list with 4 elements.

```
print(list2)
print(list2[1])
print(list2[1][0], list2[1][1], list2[1][2], list2[1][3])
```

```
[None, [1, 'xy', True, 'abc']]
[1, 'xy', True, 'abc']
1 xy True abc
```

With negative indices, elements can be accessed from the end of a list. For example, -1 refers to the last element.

```
print(list1)
print(list1[-1], list1[-3])
```

```
[1, 'xy', True, 'abc']
abc xy
```

Accessing Multiple Elements

Index ranges can be accessed using the `[start:stop:step]` syntax. `start` is the first element to be included, `stop` is *the first element not included*, and `step` defines the increment.

Access Operator	Slice
list[start:stop]	Elements from start to stop - 1
list[:]	All elements of the list
list[start:]	Elements from start to the end of the list
list[:stop]	Elements from the beginning of the list up to stop - 1
list[::3]	Every third element

Negative values for `start`, `stop`, or `step` select elements in reverse order.

Access Operator	Slice
list[-1]	the last element of the list
list[-2:]	the last two elements of the list
list[:-2]	all elements except the last two
list[::-1]	all elements in reverse order
list[1::-1]	the first two elements in reverse order
list[:-3:-1]	the last two elements in reverse order
list[-3::-1]	all elements except the last two in reverse order

(Examples by Greg Hewgill, licensed under CC BY-SA 4.0, available on stackoverflow, 2009)

Strings

Slices can also be applied to strings.

```
print('Ich bin ein string'[::2])
print('Hallo Welt'[0:6])
print('abc'[::-1])
```

```
Ihbnensrn
Hallo
cba
```

5.1.2 List Methods

Various methods are defined for the list type.

Determining Elements

- `list.index(x, start, stop)` returns the index position of the first element `x`. The optional arguments `start` and `stop` allow you to limit the search range.
- `list.count(x)` returns the frequency of `x` in the list.
- `list.reverse()` reverses the order of the list elements (this **modifies the list**!).
- `list.sort(reverse = False)` sorts the list, with the optional argument `reverse = True` for descending order (this **modifies the list**!). The data types within the list must be sortable (i.e., all elements are numeric or strings).

```
print(list1)

list1.reverse()
print(list1)

# True is counted as 1
print("True is counted as 1:", list1.index(1), list1.count(1))
```

```
[1, 'xy', True, 'abc']
['abc', True, 'xy', 1]
True is counted as 1: 1 2
```

Inserting Elements

- `list.append(x)` adds a single element to the end of the list.
- `list.extend(iterable)` adds all elements from the given iterable to the end of the list. The iterable can be a list, a tuple, a set, or a dictionary.
- `list.insert(i, x)` inserts element `x` at position `i`.

```
print(list1, "\n")

list1.append('Hello')
list1.extend(['Hello', 'World!'])
list1.insert(2, '12345')

print(list1)
```

```
['abc', True, 'xy', 1]

['abc', True, '12345', 'xy', 1, 'Hello', 'Hello', 'World!']
```

Removing Elements

- `list.remove(x)` removes *the first* occurrence of x in the list and raises a `ValueError` if x is not in the list.
- `list.pop(i)` removes the element at index position i. If no index is specified, the last element is removed. The method `list.pop(i)` returns the removed element.
- `list.clear()` removes all elements from a list.

```
list1.remove('Hello')
print(list1)

list1.pop(2)
```

```
['abc', True, '12345', 'xy', 1, 'Hello', 'World!']

'12345'
```

Copying Lists and List Elements

In Python, lists do not store data directly; instead, they consist of references (pointers) to the memory locations of the elements they contain. When a list is created by assigning it to another list, the elements themselves are **not copied**; instead, both lists refer to the **same memory locations**.

```
# Copying by assignment
list1 = [1, 'xy', True, text_variable]
print("list1:", list1, "\n")
list2 = list1

## Changing an element in list2
list2[0] = 'ABC'
print("list1 also changes because of the assignment to list2:", list1, "\n")
```

```
list1: [1, 'xy', True, 'abc']

list1 also changes because of the assignment to list2: ['ABC', 'xy', True, 'abc']
```

To copy a list and create a new object in memory, the method `list.copy()` can be used. Using the slice operator `[:]` also creates a new list in memory.

```
# Using the list.copy() method
list1 = [1, 'xy', True, text_variable]
list2 = list1.copy()

# Modify an element in list2
list2[0] = 'ABC'
print("list1 remains unchanged by the assignment to list2:", list1, "\n")

# Using the slice operator
list1 = [1, 'xy', True, text_variable]
list2 = list1[:]

# Modify an element in list2
list2[0] = 'ABC'
print("list1 remains unchanged by the assignment to list2:", list1)
```

```
list1 remains unchanged by the assignment to list2: [1, 'xy', True, 'abc']

list1 remains unchanged by the assignment to list2: [1, 'xy', True, 'abc']
```

Copying list items is not a problem in this regard.

```
# Using the slice operator
list1 = [1, 'xy', True, text_variable]
list2 = list1[0:2]

# Changing an element in list2
list2[0] = 'ABC'
print("list1 remains unchanged by the assignment in list2:", list1)
```

```
list1 remains unchanged by the assignment in list2: [1, 'xy', True, 'abc']
```

To check whether two objects share the same memory space, you can compare their object IDs using the `id()` function or use the `is` / `is not` operators, which internally call `id()`.

```
list1 = [1, 'xy', True, text_variable]
list2 = list1

print("ID of list1:", id(list1))
print("ID of list2:", id(list2))
print("Is ID of list1 equal to ID of list2:", list1 is list2)
```

```
ID of list1: 139879980071104
ID of list2: 139879980071104
Is ID of list1 equal to ID of list2: True
```

Warning 4: Identity vs. Value Equality

The `is` operator checks the identity of two objects and differs from the logical operator `==`, which checks for value equality. Since `list1` and `list2` contain the same elements, both value equality and identity are relevant. The difference between identity and value equality can be illustrated with a single value (the code example suppresses a syntax warning).

```
# Value equality
print(1 == 1.0)
print(list1 == list2, "\n")

# Identity
print(1 is 1.0)
print(list1 is list2)
```

```
True
True

False
True
```

5.1.3 List Exercises

1. Determine the position of the value 0 in the list `numbers = [34, 12, 0, 67, 23]`. Remove the value from the list and print the list sorted in ascending order.
2. Now, using the access operator `[]`, print the index positions 1 and 3 of the sorted list `numbers`.
3. Create a list containing the numbers 1 through 10.

- Use the slice operator to print all even numbers.
- Use the slice operator to print all odd numbers in descending order.

4. Create a list `weekdays` containing the seven days of the week. Use the slice operator to create a new list `weekend` with the weekend days. Remove the weekend days from the `weekdays` list.
5. 4-day week: Use list methods to remove Friday from the `weekdays` list and add it to the `weekend` list before Saturday.

Tipp 11: Example Solution: Lists

1. Exercise

```
# 1.
numbers = [34, 12, 0, 67, 23]
target_index = numbers.index(0)
print(target_index)

numbers.remove(0)
numbers.sort()
print(numbers)
```

```
2
[12, 23, 34, 67]
```

2. Exercise

```
# 2.
print(f"Value of the number at index position 1: {numbers[1]}")
print(f"Value of the number at index position 3: {numbers[3]}")
```

```
Value of the number at index position 1: 23
Value of the number at index position 3: 67
```

3. Exercise

```
# 3.
lst = list(range(1, 11))
print(lst[1::2])
print(lst[-2::-2])
```

```
[2, 4, 6, 8, 10]
[9, 7, 5, 3, 1]
```

4. Exercise

```
# 4.
# Task: Days of the week
weekdays = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
weekend = weekdays[5:7]  # you could also use weekdays[-2:]
print("Weekend:", weekend)

weekdays.pop(-1)
weekdays.pop(-1)
print(weekdays)
```

```
Weekend: ['Saturday', 'Sunday']
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']
```

5. Exercise

```
# 5.
weekend.insert(0, weekdays.pop(4))
print(weekdays)
print(weekend)
```

```
['Monday', 'Tuesday', 'Wednesday', 'Thursday']
['Friday', 'Saturday', 'Sunday']
```

Sample Solution by Marc Sönnecken

5.2 Tuples

Tuples are very similar to lists, but tuples are immutable data objects. This means that the elements of an existing tuple cannot be changed or removed. You also cannot add new elements to a tuple.

Tuples are created using parentheses `()`, and elements are separated by commas `,`. A tuple with a single value is created with a comma in the form `(value, )`. Accessing elements of a tuple is possible using the slice operator `[start:stop:step]`. Tuples can be concatenated using the `+` and `*` operators.

```
tuple1 = (2, 7.8, 'Fire', True, text_variable)
tuple2 = (1, )

print(tuple1)
```

```
print(tuple1[2:4])
print(tuple1[::2])
print(tuple1[-1])
print(tuple1[2:4] + tuple2)
print(3 * tuple2)
```

```
(2, 7.8, 'Fire', True, 'abc')
('Fire', True)
(2, 'Fire', 'abc')
abc
('Fire', True, 1)
(1, 1, 1)
```

5.2.1 Copying Tuples

Tuples behave differently from lists when it comes to copying. Tuples do not have a `.copy()` method. Using slicing `[:]` allows you to create a copy, but both tuples initially reference the same memory. When a tuple is reassigned, Python, as with any object, creates a new object in memory.

```
# Copying by assignment
tuple1 = (1, 2, 3)
tuple2 = tuple1

## Reassigning values of tuple1
tuple1 = (4, 5, 6)
print(f"The values stored in tuple2 remain unchanged:\n{tuple1} {tuple2}\n")

# Copying using slice operator
tuple1 = (1, 2, 3)
tuple2 = tuple1[:]
print(tuple2 is tuple1)

## Reassigning values of tuple1
tuple1 = (4, 5, 6)
print(tuple1, tuple2)
```

```
The values stored in tuple2 remain unchanged:
(4, 5, 6) (1, 2, 3)
```

```
True
(4, 5, 6) (1, 2, 3)
```

5.3 Sets

In Python, sets can be created from a list or a tuple using the `set()` function, or directly with curly braces `{}` (an empty set can only be created with `set()`, because `{}` creates an empty dictionary). Sets are unordered collections, so their elements do not have a specific order.

```python
my_list = [1, 1, 5, 3, 3, 4, 2, 'a', 123, 1000, ('tuple', 5)]
print("The list object as a set:\n", set(my_list))

my_set = {1, 2, 3, 4, 5, 1000, ('tuple', 5), 'a', 123}
print("A set can also be created using curly braces:", my_set, sep="\n")
```

```
The list object as a set:
 {1, 2, 3, 4, 5, 1000, ('tuple', 5), 'a', 123}
A set can also be created using curly braces:
{1, 2, 3, 4, 5, 1000, ('tuple', 5), 'a', 123}
```

Sets can be used, for example, for comparison operations.

```python
set_a = set('Python')
set_b = set('is awesome')

# unique characters in a
print("Set a:", set_a)

# characters in a but not in b
print("Set a - b:", set_a - set_b)

# characters in a or b
print("Set a | b:", set_a | set_b)

# characters in a and b
print("Set a & b:", set_a & set_b)

# characters in a or b, but not in both (XOR)
print("Set a ^ b:", set_a ^ set_b)
```

```
Set a: {'y', 'P', 'o', 'n', 't', 'h'}
Set a - b: {'y', 'P', 'n', 't', 'h'}
Set a | b: {'i', 'y', 'P', 'o', ' ', 'e', 'n', 'a', 'm', 't', 's', 'w', 'h'}
Set a & b: {'o'}
Set a ^ b: {'i', 'y', 'P', ' ', 'e', 'a', 'm', 'n', 't', 's', 'w', 'h'}
```

5.3.1 Copying Sets

Sets behave similarly to tuples, with the difference that the `.copy()` method is defined for sets. However, the indexing operator `[]` cannot be used on sets.

```python
# Copying by assignment
set1 = {1, 2, 3}
set2 = set1
print(set1 is set2)

## Reassigning set1
set1 = {4, 5, 6}
print(f"The values stored in set2 remain unchanged:\n{set1} {set2}")

# Copying using the .copy() method
set1 = {1, 2, 3}
set2 = set1.copy()
print(set1 is set2)

## Reassigning set1
set1 = {4, 5, 6}
print(f"The values stored in set2 remain unchanged:\n{set1} {set2}")
```

```
True
The values stored in set2 remain unchanged:
{4, 5, 6} {1, 2, 3}
False
The values stored in set2 remain unchanged:
{4, 5, 6} {1, 2, 3}
```

5.4 Dictionaries

Dictionaries consist of key-value pairs. Keys can be numbers or strings, and each key can only appear once. Dictionaries are defined using curly braces `{}`. Keys and their corresponding

values are separated by a colon `:`. Values are accessed using the bracket operator `[]`, which contains the key(s). Accessing values by the positional index of keys is not possible, since numbers are interpreted as keys.

```
dictionary1 = {1: 'abc', 'b': [1, 2, 3], 'c': ('tuple', 5, 6)}
print(dictionary1, "\n")

print("Values of key 1:", dictionary1[1])
print("Values of key 'b':", dictionary1['b'])
```

```
{1: 'abc', 'b': [1, 2, 3], 'c': ('tuple', 5, 6)}

Values of key 1: abc
Values of key 'b': [1, 2, 3]
```

You can access the keys of a dictionary using the `dictionary.keys()` method, and the values using the `dictionary.values()` method.

```
print("Keys:", dictionary1.keys(), "\n")
print("Values:", dictionary1.values())
```

```
Keys: dict_keys([1, 'b', 'c'])

Values: dict_values(['abc', [1, 2, 3], ('tuple', 5, 6)])
```

5.4.1 Copying Dictionaries

Dictionaries behave like lists when copied, meaning that copying via assignment will share the same memory space between dictionaries.

```
# Copying via assignment
print("dictionary:", dictionary1, "\n")
dictionary2 = dictionary1

## Modifying an element in dictionary2
dictionary2[1] = 'ABC'
print("dictionary1 also changes due to the assignment to dictionary2:\n",
       dictionary1, "\n")

# Using the dictionary.copy() method
```

```python
dictionary1 = {1: 'abc', 'b': [1, 2, 3], 'c': ('tuple', 5, 6)}
dictionary2 = dictionary1.copy()

## Modifying an element in dictionary2
dictionary2[1] = 'ABC'
print("dictionary1 remains unchanged by the assignment to dictionary2:\n",
       dictionary1, "\n")
```

```
dictionary: {1: 'abc', 'b': [1, 2, 3], 'c': ('tuple', 5, 6)}

dictionary1 also changes due to the assignment to dictionary2:
 {1: 'ABC', 'b': [1, 2, 3], 'c': ('tuple', 5, 6)}

dictionary1 remains unchanged by the assignment to dictionary2:
 {1: 'abc', 'b': [1, 2, 3], 'c': ('tuple', 5, 6)}
```

5.5 Overview of Collection Types

Feature	Lists	Tuples	Sets	Dictionary
Description	flexible number of elements of any type	elements of any type, immutable	unordered collection, each element appears only once	mapping table of key-value pairs
Memory shared on assignment	yes	yes (but immutable)	yes (but indexing operator not applicable)	yes

Feature	Lists	Tuples	Sets	Dictionary
.copy() method defined	yes	no	yes	yes
Slice operator applicable	yes	yes	no	yes (by key)

5.6 Deleting: the `del` Keyword

To delete collections, elements, or slices, the `del` keyword can be used.

```
# Deleting a list
del list1

# Deleting a range of indices from a list
print("List before deletion:", list2)
del list2[1:3]
print("List after deletion:", list2)

# Deleting a key from a dictionary
print("Dictionary before deletion:", dictionary1)
del dictionary1[1]
print("Dictionary after deletion:", dictionary1)
```

```
List before deletion: [1, 'xy', True, 'abc']
List after deletion: [1, 'abc']
Dictionary before deletion: {1: 'abc', 'b': [1, 2, 3], 'c': ('tuple', 5, 6)}
Dictionary after deletion: {'b': [1, 2, 3], 'c': ('tuple', 5, 6)}
```

5.7 Functions

Collection types can be converted into each other.

```
dictionary = {1: 'Tomcat', 2: 'Vixen', 3: 'Doe'}
print( (list_var := list(dictionary)) )
print( (set_var := set(list_var)) )
print( (tuple_var := tuple(set_var)) )
```

```
[1, 2, 3]
{1, 2, 3}
(1, 2, 3)
```

Some practical functions can also be applied to collection types:

- `len()` returns the number of elements in a collection.
- `min()`, `max()`, `sum()` return the minimum, maximum, or sum of a collection (for dictionaries, the number of keys is counted).

5.8 Operations: Using Loops

To apply arithmetic and logical operators to the elements stored in a collection, a for-loop is used. In the following example, a list `numbers` is iterated over, the numbers it contains are squared, and each result is appended to the list `squared_numbers`. It also checks whether the squared numbers are divisible by 3 and stores the check results in the list `modulo_3`.

Code-Block 5.1

```python
numbers = list(range(1, 11))

squared_numbers = [] # the list must be created before the loop
modulo_3 = [] # create an empty list before the loop

for number in numbers:
  squared_number = number ** 2
  squared_numbers.append(squared_number)
  modulo_3.append(squared_number % 3 == 0)

print(squared_numbers)
print(modulo_3)
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
[False, False, True, False, False, True, False, False, True, False]
```

5.9 Collection Types Exercises

1. Modify the program code in Code-Block 5.1 so that only square numbers divisible by 3 are stored.

2. Speed Conversion
   Create a script that generates a conversion table for speeds. The following conditions should be observed:

- The conversion should be from km/h to m/s.
- The start and end values should be freely selectable in km/h, with both being integers.
- The number of conversion points should be definable, and the intermediate steps (in km/h) should always be output as whole numbers.

Tip: In your script, you can use the input() function to capture values from the user.

3. Sorting: Given the list `my_list = list(range(9, 0, -1))`. It should be sorted using for-loops (for example, using the Bubble Sort algorithm).

Tipp 12: Sample Solution for Collection Types Exercises

1. Square numbers divisible by 3

```
numbers = list(range(1, 11))

square_numbers = [] # the list must be created before the loop
divisible_by_3 = [] # create an empty list before the loop

for number in numbers:
    square = number ** 2
    if square % 3 == 0:
        square_numbers.append(square)

print(square_numbers)
```

```
[9, 36, 81]
```

2. Conversion of Speeds

```python
# Free input
## start = int(input("Enter starting value in kilometers per hour: "))
## end = int(input("Enter ending value in kilometers per hour: "))
## steps = int(input("Enter number of steps to output: "))

# Fixed values for the solution
start = 5
end = 107
steps = 8

# Create list for km/h
step_size = (end - start) / (steps - 1)
km_list = []
for i in range(steps):
    km_list.append(round(start + i * step_size))

# Conversion
# meters = 1000 * kilometers
# seconds = hours * 60 * 60
m_per_s_list = []
for value in km_list:
    m_per_s_list.append(round((value * 1000) / (60 * 60), 2))

# Output
print(f"Step size: {step_size:.2f}")
print("Kilometers per hour")
print(km_list)
print("Meters per second")
print(m_per_s_list)
```

```
Step size: 14.57
Kilometers per hour
[5, 20, 34, 49, 63, 78, 92, 107]
Meters per second
[1.39, 5.56, 9.44, 13.61, 17.5, 21.67, 25.56, 29.72]
```

3. Sorting: Bubble Sort Algorithm

```python
# static list, text output
my_list = list(range(9, 0, -1))

if len(my_list) > 1:

    print("Initial list\t\t  :", my_list)

    # outer loop
    step = 0
    for i in range(len(my_list) - 1):

        # inner loop
        for j in range(len(my_list) - 1):
            if my_list[j] > my_list[j + 1]:
                my_list[j], my_list[j + 1] = my_list[j + 1], my_list[j]

        step += 1
        print("List after step", step, ":", my_list)

    print("\nSorted list:", *my_list) # * suppresses commas between list elements

else:
    print("The list must contain at least two elements!")
```

```
Initial list          : [9, 8, 7, 6, 5, 4, 3, 2, 1]
List after step 1 : [8, 7, 6, 5, 4, 3, 2, 1, 9]
List after step 2 : [7, 6, 5, 4, 3, 2, 1, 8, 9]
List after step 3 : [6, 5, 4, 3, 2, 1, 7, 8, 9]
List after step 4 : [5, 4, 3, 2, 1, 6, 7, 8, 9]
List after step 5 : [4, 3, 2, 1, 5, 6, 7, 8, 9]
List after step 6 : [3, 2, 1, 4, 5, 6, 7, 8, 9]
List after step 7 : [2, 1, 3, 4, 5, 6, 7, 8, 9]
List after step 8 : [1, 2, 3, 4, 5, 6, 7, 8, 9]

Sorted list: 1 2 3 4 5 6 7 8 9
```

(Arnold (2023d))

6 Defining Your Own Functions

Defining your own functions opens up a wide range of possibilities in Python:

- Complex programs can be called and executed with a single line of code.
- Functions can be called practically any number of times and, through the use of parameters and control flow statements, can flexibly respond to changing conditions.
- Functions make code shorter and more readable. Additionally, there is only one place that needs to be updated when changes are necessary.

6.1 Syntax

The keyword `def` starts the function definition. It is followed by the function name and the function parameters, which are enclosed in parentheses `()`. The function header ends with a colon `:`. The function's code block is indented. Every function returns a value, which is passed back to the calling location using the `return` keyword. `return` ends the function's execution, even if it is not at the end of the code block.

```
def function_name(parameter1, parameter2):
    code_block
    return return_value
```

In order for the function to be executed, the defined function name must be called. The function contains another statement after the keyword `return`, which is no longer executed.

```
# Example 1: Sum of Squares

# Definition of a function to calculate the sum of squares of two arguments
def sum_squares(a, b):
    print('Argument a:', a)
    print('Argument b:', b)
    print(18 * '=')
    total = a**2 + b**2
    return total
```

```
    print("Statements after the 'return' keyword are no longer executed.")

print(sum_squares(6, 7))
```

```
Argument a: 6
Argument b: 7
==================
85
```

The return value can be stored in a variable.

```
result = sum_squares(6, 7)
print(result)
```

```
Argument a: 6
Argument b: 7
==================
85
```

6.2 Optional Parameters

Optional parameters can be used to control the execution of a program. Optional parameters must be defined after the required parameters. In this example, the print output of the function is controlled using the parameter `output`.

```
# Example 2: optional arguments

# Define a function to calculate the sum of the squares of two arguments
def sum_squares(a, b, output=False):
    if output:
        print('Value of argument a:', a)
        print('Value of argument b:', b)
        print(18 * '=')
    total = a**2 + b**2
    return total

print(sum_squares(42, 7), "\n")
print(sum_squares(42, 7, output=True))
```

```
1813

Value of argument a: 42
Value of argument b: 7
==================
1813
```

If there are multiple optional parameters, arguments are assigned positionally or using the keyword.

```python
# Example 3: multiple optional arguments

# Define a function to calculate the sum of powers of two arguments
def sum_powers(a, b, p=2, output=False):
    if output:
        print('Argument a:', a)
        print('Argument b:', b)
        print('Argument p:', p)
        print(18 * '=')
    total = a**p + b**p
    return total

# positional passing
print(sum_powers(42, 7, 3, True), "\n")

# keyword passing
print(sum_powers(42, 7, output=True, p=4))
```

```
Argument a: 42
Argument b: 7
Argument p: 3
==================
74431

Argument a: 42
Argument b: 7
Argument p: 4
==================
3114097
```

6.3 Return Value(s)

In Python, functions can only have a single return value. However, multiple values can be separated by commas. Python returns these as a tuple.

```
# Example 4: multiple return values

# Definition of a function to calculate the sum of powers of two arguments
def sum_powers(a, b, p = 2, output = False):
    if output:
        print('Argument a:', a)
        print('Argument b:', b)
        print('Argument p:', p)
        print(18 * '=')
    total = a**p + b**p
    return a, b, total

result = sum_powers(2, 7, output = False, p = 4)
print(result, type(result))
```

```
(2, 7, 2417) <class 'tuple'>
```

The slice operator allows you to select a specific return value.

```
print(result[2])

sum_of_powers = sum_powers(2, 7, output=False, p=4)[2]
print(sum_of_powers, type(sum_of_powers))
```

```
2417
2417 <class 'int'>
```

6.4 Exercises Defining Functions

1. Palindrome
   Write a function `is_palindrome()` that checks whether a given string is a palindrome.

Note: A palindrome is a string that reads the same forwards and backwards, such as 'Anna', 'Otto', 'racecar'. Palindromes do not have to consist only of letters; they can also contain numbers or a combination of letters and numbers, like '345g543'.

2. Fibonacci Sequence

Develop a function `fibonacci(n)` that generates the first n numbers of the Fibonacci sequence and returns them as a list. The Fibonacci sequence starts with 0 and 1, and each subsequent number is the sum of the two preceding numbers.

3. Encryption

In geocaching, encrypted messages are often used as puzzles. One common encryption logic is:

- A -> Z
- B -> Y
- C -> X
- …

Write a function `encrypt(string)` that takes a string as input and returns an encrypted string. How can you most easily decrypt an encrypted string?

4. Temperature Converter

Develop a function `convert_temperature(value, from_unit, to_unit)` that converts a temperature from one unit to another. The function should use the following parameters:

- value: The temperature value to be converted.
- from_unit / to_unit: The unit of the input and target value as a string. Possible values are 'C' for Celsius, 'F' for Fahrenheit, and 'K' for Kelvin.

The following conversion formulas between units apply:

- Celsius to Fahrenheit: F = C * 9/5 + 32
- Fahrenheit to Celsius: C = (F - 32) * 5/9
- Celsius to Kelvin: K = C + 273.15
- Kelvin to Celsius: C = K - 273.15
- Fahrenheit to Kelvin: K = (F - 32) * 5/9 + 273.15
- Kelvin to Fahrenheit: F = (K - 273.15) * 9/5 + 32

Sample Solution: Defining Functions Exercises

1. Palindrome Exercise

```python
# 1.
def is_palindrome(text):
    text = text.lower()
    return text == text[::-1]

# string to test
test_string = "racecar"
if is_palindrome(test_string):
    print(f"Yes, {test_string} is a palindrome")
else:
    print(f"No, {test_string} is not a palindrome")
```

```
Yes, racecar is a palindrome
```

2. Exercise Fibonacci

```python
# 2.
def fibonacci(n):

    # the special cases must be considered:
    if n <= 0:
        return []
    elif n == 1:
        return [0]

    # after considering the two special cases, the list always starts as follows:
    fibonacci_sequence = [0, 1]

    while len(fibonacci_sequence) < n:
        next_number = fibonacci_sequence[-1] + fibonacci_sequence[-2]
        fibonacci_sequence.append(next_number)

    return fibonacci_sequence

fibonacci(8)
```

```
[0, 1, 1, 2, 3, 5, 8, 13]
```

3. Exercise Encryption

6.5 Using Lists

```python
# 3.
def encrypt(text):

    # in six variables, the normal and the reversed alphabet as well as the digits 0-9 are
    abc = "abcdefghijklmnopqrstuvwxyz"
    abc_2 = "zyxwvutsrqponmlkjihgfedcba"
    ABC = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    ABC_2 = "ZYXWVUTSRQPONMLKJIHGFEDCBA"
    digits = "0123456789"
    digits_2 = "9876543210"

    new_string = ""

    for char in text:
        if char in ABC: # uppercase letters
            position = ABC.index(char)
            new_character = ABC_2[position]

            new_string += new_character
        elif char in abc: # lowercase letters
            position = abc.index(char)
            new_character = abc_2[position]

            new_string += new_character
        elif char in digits: # digits
            position = digits.index(char)
            new_character = digits_2[position]

            new_string += new_character
        else: # other characters are kept unchanged
            new_string += char

    return new_string

# message to be encrypted:
secret_message = 'This is the secret: <*)))><'
encoded_text = encrypt(secret_message)
print(encoded_text)

# the simplest way to decrypt the encrypted message is to apply the same function again to
decoded_text = encrypt(encoded_text)
print(decoded_text)
```

```
Gsrh rh gsv hvxivg: <*)))><
This is the secret: <*)))><
```

6.6 Using ord() and chr()

```python
def encrypt(a):
    a = list(a)
    # print(a)
    b = []  # output list

    for i in a:
        # print(i, ord(i))
        b.append(ord(i))
    # print(b)

    # invert: a = 97, z = 122, A = 65, Z = 90, 0 = 48, 9 = 57
    # max - value + min

    for i in range(len(b)):
        if b[i] >= 65 and b[i] <= 90:  # uppercase letters
            b[i] = chr(90 - b[i] + 65)
        elif b[i] >= 97 and b[i] <= 122:  # lowercase letters
            b[i] = chr(122 - b[i] + 97)
        elif b[i] >= 48 and b[i] <= 57:  # digits
            b[i] = chr(57 - b[i] + 48)
        else:  # other characters
            b[i] = chr(b[i])

    return ''.join(b)

print(encoded_text)
print(encrypt(a=encoded_text))
```

```
Gsrh rh gsv hvxivg: <*)))><
This is the secret: <*)))><
```

4. Temperature Converter

```python
# 4.
def convert_temperature(value, from_unit, to_unit):

    # if statements to check which unit the value to be converted is in.
    # Depending on which branch is triggered, the value is converted
    # to the target unit using additional if statements

    # if statement for original value in Celsius:
    if from_unit == "C":
        if to_unit == "F":
            return value * 9/5 + 32
        elif to_unit == "K":
            return value + 273.15
        elif to_unit == "C":
            return value

    # if statement for original value in Fahrenheit:
    if from_unit == "F":
        if to_unit == "C":
            return (value - 32) * 5/9
        elif to_unit == "K":
            return (value - 32) * 5/9 + 273.15
        elif to_unit == "F":
            return value

    # if statement for original value in Kelvin:
    if from_unit == "K":
        if to_unit == "C":
            return value - 273.15
        elif to_unit == "F":
            return (value - 273.15) * 9/5 + 32
        elif to_unit == "K":
            return value

    # return None for invalid units
    return None

test_temp = convert_temperature(37, "C", "F")
print(f"37°C in Fahrenheit is: {test_temp}°F")
```

```
37°C in Fahrenheit is: 98.6°F
```

Sample solution by Marc Sönnecken and Maik Poetzsch

(Arnold (2023b))

7 Reading and writing files

Maya and Hans each rolled a die six times and recorded their results in a `.txt` file. We want to evaluate the files using Python in order to determine which of the two achieved the higher total number of pips.

Data	File name
Dice results Maya	dice-maya.txt
Dice results Hans	dice-hans.txt

7.1 File objects

In order to access a file with Python, it must be opened for reading or writing. For this purpose, Python provides the function open. It takes two arguments—the file path and the access mode—and returns a file object. The contents of the file are then read from this file object.

7.1.1 File path

The local file path is specified relative to the current working directory.

```
path_maya = "01-daten/dice-maya.txt"
path_hans = "01-daten/dice-hans.txt"
```

Tipp 13: Determine and change the working directory in Python

The path of the current working directory can be obtained using the `os` module via `os.getcwd()` (here without output). The working directory can be changed if necessary using `os.chdir('new_path')`. You can recognize the correct formatting of the path from the output of `os.getcwd()`.

```
import os
print(os.getcwd())
```

The importing of modules is covered in a later chapter.

7.1.2 Access mode

Among others, the following options are available as access modes:

Mode	Description
`r`	read access
`w`	write access, file is overwritten
`x`	creates the file, error if the file already exists
`a`	write access, contents are appended
`b`	binary mode (e.g. for images)
`t`	text mode, default

The access modes can also be combined. Further information can be found in the documentation. Unless files are accessed in binary mode, the `open()` function returns the file contents as a string.

In read mode, a data object is created.

```
data_maya = open(path_maya, mode='r')
print(data_maya)
```

```
<_io.TextIOWrapper name='01-daten/dice-maya.txt' mode='r' encoding='UTF-8'>
```

When the data object `data_maya` is passed to the `print()` function, Python outputs the class of the object, in this case `_io.TextIOWrapper`. This class comes from the `io` module and is responsible for reading and writing text files. In addition, the file object's attributes—such as the file path, access mode, and file encoding—are displayed (see Beispiel 2). If the encoding is not automatically detected as UTF-8, it can be explicitly specified using the argument `encoding = 'UTF-8'`.

```
data_maya = open(path_maya, mode='r', encoding='UTF-8')
print(data_maya)
```

```
<_io.TextIOWrapper name='01-daten/dice-maya.txt' mode='r' encoding='UTF-8'>
```

Hinweis 2: Determining an Object's Attributes

The function `dir(object)` can be used to list the available attributes of an object. However, it also includes inherited attributes and methods from the object's class, so the output is often quite extensive. For example, for the integer 1:

```
print(dir(1))
```

```
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__di
```

To limit the output to attributes, the following function can be used:

```
object_var = 1

attributes = [attr for attr in dir(object_var) if not callable(getattr(object_var, attr))]
print(attributes)
```

```
['__doc__', 'denominator', 'imag', 'numerator', 'real']
```

Attributes enclosed with double underscores are reserved for Python and not intended for the user. The following function removes attributes with double underscores from the output:

```
my_object = 1

attributes = [attr for attr in dir(my_object) if not (callable(getattr(my_object, attr)) or
print(attributes)
```

```
['denominator', 'imag', 'numerator', 'real']
```

In the case of an integer, attributes (to distinguish from floating-point numbers using enclosing parentheses) can be accessed as follows:

```
(1).numerator
```

```
1
```

If we look at the attributes of the file object 'data_maya', attributes with a single leading underscore stand out.

```
object_ = data_maya

attributes = [attr for attr in dir(object_) if not (callable(getattr(object_, attr)) or att
print(attributes)
```

```
['_CHUNK_SIZE', '_finalizing', 'buffer', 'closed', 'encoding', 'errors', 'line_buffering',
```

These are attributes that should not be accessed by the user (Further information can be found here). The following code prints all attributes without leading underscores:

```
object_instance = data_maya

attributes = [attr for attr in dir(object_instance) if not (callable(getattr(object_instanc
print(attributes)
```

```
['buffer', 'closed', 'encoding', 'errors', 'line_buffering', 'mode', 'name', 'newlines', 'w
```

File attributes can be accessed using the corresponding commands.

```
print(f"File path: {data_maya.name}\n"
      f"File name: {os.path.basename(data_maya.name)}\n"
      f"File closed: {data_maya.closed}\n"
      f"Access mode: {data_maya.mode}\n"
      f"Encoding: {data_maya.encoding}")
```

```
File path: 01-daten/dice-maya.txt
File name: dice-maya.txt
File closed: False
Access mode: r
Encoding: UTF-8
```

Tipp 14: Fallback Option

In data analysis, specialized packages like NumPy or Pandas are typically used. These make reading files much easier compared to base Python. Nevertheless, it is useful to be familiar with base Python methods for reading files. Using the `open()` function works almost always—it's a reliable fallback option.

7.1.3 Print File Content

To print the content of a file, the data object can be iterated line by line using a loop. (The file `dice-maya` only has one line.)

```
i = 0
for line in data_maya:
    print(f"Content of line {i}, with {len(line)} characters:")
    print(line)
    i += 1
```

```
Content of line 0, with 28 characters:
"5", "6", "2", "1", "4", "5"
```

However, this approach is not very practical for larger files. Using the `print()` function to output individual lines can be helpful to check the exact formatting of the string. In this case, Maya has placed her data in quotation marks and separated them with commas.

7.2 Reading Files

To read the entire content of a file, you can use the method file_object.read(). The method has an optional parameter `.read(size)`. The `size` is passed as an integer, and the corresponding number of characters (or bytes in binary mode) will be read, up to the end of the file. The `size` parameter is useful for checking the formatting of a large file's content while limiting the output to a manageable number of characters.

```
eyes_maya = data_maya.read()

print(f"len(eyes_maya): {len(eyes_maya)}\n\n"
      f"Content of the file eyes_maya:\n{eyes_maya}")
```

```
len(eyes_maya): 0

Content of the file eyes_maya:
```

This obviously didn't work because the read file content was empty! The reason is that when reading (and writing) a file, the file pointer moves through the file. After the file `data_maya` was output line by line in Kapitel 7.1.3, the file pointer is at the end of the file.

Warning 5: File Pointer in Python

When a file is read line by line or with the `.read()` method, the file pointer moves by the specified number of characters or to the end of the file. For example, if a dataset `data` is opened and the first three characters are read using `data.read(3)`, the file pointer moves from index position 0 to index position 3 (or is just before it).

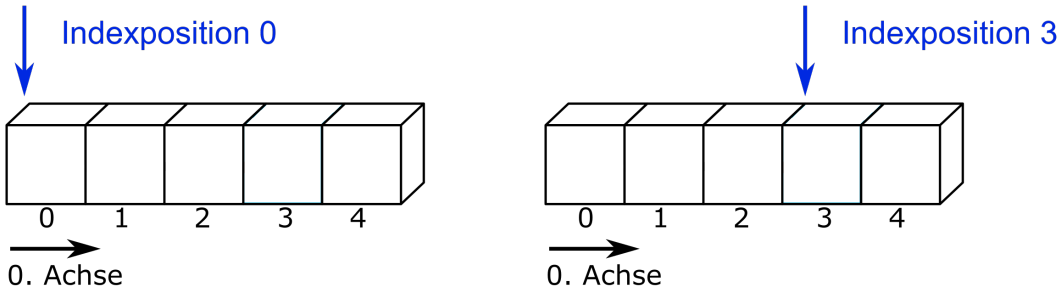


Abbildung 7.1: Movement of the file pointer when reading three characters

The method `data.tell()` returns the current position of the file pointer.
The method `data.seek(offset, whence = 0)` sets the pointer to a specific position. It accepts the argument `offset` (displacement) and the optional argument `whence` (from where), which defaults to 0 (start of the file). For **binary mode** access (`open(path, mode='rb')`), `whence` can also take the values 1 (current position) or 2 (end of the file).

- `data.seek(0, 0)` refers to the start of the file
- `data.seek(0, 1)` refers to the current position in the file
- `data.seek(0, 2)` refers to the end of the file
- `data.seek(-3, 2)` refers to the third character before the end of the file

If the file pointer is moved back to the start using `data_object.seek(0)`, reading the file succeeds.

```
print(f"File pointer position before resetting to 0: {data_maya.tell()}")

data_maya.seek(0);
print(f"File pointer position after resetting to 0: {data_maya.tell()}")

eyes_maya = data_maya.read()
```

```
print(f"Content of the object eyes_maya:\n{eyes_maya}")
```

```
File pointer position before resetting to 0: 28
File pointer position after resetting to 0: 0
Content of the object eyes_maya:
"5", "6", "2", "1", "4", "5"
```

Print the numbers at the second and third positions (i.e., 6 and 2) from the data object `data_maya` using the `.seek()` and `.read()` methods.

Tipp 15: Example Solution: Moving the File Pointer

```
data_maya.seek(6, 0)
print(data_maya.read(1))

data_maya.seek(data_maya.tell() + 4, 0)
print(data_maya.read(1))
```

```
6
2
```

To add Maya's dice results, the numbers must be extracted and converted to integers, because in text mode a string is always returned.

```
print(type(eyes_maya))
```

```
<class 'str'>
```

Here, the leading and trailing quotation marks are removed using `str.strip("")`, and then the string is split into a list using `str.split('", "')`. Afterwards, the list elements are converted to integers and summed. (String manipulation methods will be covered in detail in the next section.)

```
print(f"eyes_maya:\n{eyes_maya}")

eyes_maya = eyes_maya.strip('"')
print(f"\neyes_maya.strip('\"'):\n{eyes_maya}")

eyes_maya = eyes_maya.split('", "')
print(f"\neyes_maya.split('\", \"'):\n{eyes_maya}")
```

```
eyes_maya_int = []
for i in eyes_maya:
    eyes_maya_int.append(int(i))

print(f"\neyes_maya_int:\n{eyes_maya_int}\n\nTotal eyes: {sum(eyes_maya_int)}")
```

```
eyes_maya:
"5", "6", "2", "1", "4", "5"

eyes_maya.strip('"'):
5", "6", "2", "1", "4", "5

eyes_maya.split('", "'):
['5', '6', '2', '1', '4', '5']

eyes_maya_int:
[5, 6, 2, 1, 4, 5]

Total eyes: 23
```

Closing a File

After accessing a file, it must be closed to free it up for other programs.

```
data_maya.close()
```

> **Warning 6: Writing Operations with Python**
>
> Closing a file is especially important when performing write operations on data objects. Otherwise, it may happen that content written with `data_object.write()` is not fully saved to disk. See the documentation for more details.

7.3 Task: Reading Files

What dice score did Hans achieve?

Tipp 16: Sample Solution: Dice Score Comparison

```
# First, read the file:
data_hans = open(path_hans, mode='r', encoding='UTF-8')
dice_hans = data_hans.read()
print(dice_hans)
# Here, we notice that Hans put his name at the start of his list.
# This string must be removed before summing the values!

dice_hans = dice_hans.strip('"Hans", ')
dice_hans = dice_hans.strip('"')
dice_hans = dice_hans.split('", "')
print(dice_hans)
# The print output shows that the list is now correctly cleaned.
# It contains only integer values, which can be summed.

# Create a new (empty) list for Hans' dice rolls:
dice_hans_int = []
for i in dice_hans:
    dice_hans_int.append(int(i))

print(f"Total dice score of Hans: {sum(dice_hans_int)}")
```

```
"Hans", "3", "5", "1", "3", "2", "5"
['3', '5', '1', '3', '2', '5']
Total dice score of Hans: 19
```

Solution template by Marc Sönnecken.

7.4 Interpreting Data

Datasets typically exist in at least a two-dimensional form, meaning the data is organized into rows and columns. Additionally, datasets usually contain different data types. The function `open(file)` returns a file object, which can be read as a string using methods such as `file_object.read()`. To process the data meaningfully, it is therefore necessary to correctly interpret the string and distinguish data from delimiters.

Python provides a variety of string methods for working with strings. Some of these methods are exemplified in this chapter. String methods are typically referenced with a leading 'str' in the form `str.method()`.

For example, a file containing the population of European countries will be read.

Data	Filename
European Population	einwohner_europa_2019.csv

To get an overview of the file's structure, the first three lines of the file are printed. The file can be iterated line by line using a for-loop, which is stopped using the `break` keyword when the loop variable reaches the value 3. Another option is to use the method `file_object.readline()`, which reads a single line. Here, the number of loop iterations is controlled by the loop variable with `for i in range(3):`.

7.5 for-loop with break

```
file_path = "01-daten/einwohner_europa_2019.csv"
population_file = open(file_path, 'r')

# view the first 3 lines
i = 0
for line in population_file:

  print(line)
  i += 1
  if i == 3:
    break

# close the file
population_file.close()
```

```
GEO,Value

Belgien,11467923

Bulgarien,7000039
```

7.6 Method `file_object.readline()`

The method `file_object.readline()` can be used to read a single line from a file.

```
file_path = "01-daten/einwohner_europa_2019.csv"
population_file = open(file_path, 'r')

for i in range(3):
    print(population_file.readline())

# Close the file
population_file.close()
```

```
GEO,Value

Belgien,11467923

Bulgarien,7000039
```

The file has two columns. The first column contains country names, the second column contains values. Commas are used as separators. The first row contains the column headers.

In the previous section, we learned the method `file_object.read()`, which reads an entire file as a string. First, the file is read into the variable `population` using `file_object.read()` and then closed.

```
file_path = "01-daten/einwohner_europa_2019.csv"
file_object_population = open(file_path, 'r')

population = file_object_population.read()
print(population)

# Close the file
file_object_population.close()
```

```
GEO,Value
Belgien,11467923
Bulgarien,7000039
Tschechien,10528984
Daenemark,5799763
Deutschland einschliesslich ehemalige DDR,82940663
Estland,1324820
Irland,4904240
Griechenland,10722287
Spanien,46934632
```

```
Frankreich,67028048
Kroatien,4076246
Italien,61068437
Zypern,875898
Lettland,1919968
Litauen,2794184
Luxemburg,612179
Uganda,-1
Ungarn,9772756
Malta,493559
Niederlande,17423013
Oesterreich,8842000
Polen,37972812
Portugal,10276617
Rumaenien,19405156
Slowenien,2080908
Slowakei,5450421
Finnland,5512119
Schweden,10243000
Vereinigtes Koenigreich,66647112
```

Afterwards, the read-in data can be split line by line using the `str.split('\n')` method. `'\n'` is passed as the argument to indicate the line break. The method returns a list.

```
population_list_by_line = population.split("\n")
print(population_list_by_line[0:3])
```

```
['GEO,Value', 'Belgien,11467923', 'Bulgarien,7000039']
```

The list contains the column headers at the index position. These can be removed from the list using the method `list.pop(index)` and simultaneously stored in a new object.

```
column_names = population_list_by_line.pop(0)
column_names = column_names.split(',')
print(f"Header of column 0: {column_names[0]}\tHeader of column 1: {column_names[1]}")
```

```
Header of column 0: GEO Header of column 1: Value
```

Afterwards, the list can be split into countries and values using the `str.split(',')` method. However, the process fails with an error. The error is caught in the following code block using exception handling. In addition to the error message, the list entry that caused it and its index position are also displayed.

```
# Create empty lists before the loop
geo = []
population = []

try:
    for line in population_list_by_line:
        entry = line.split(',')
        geo.append(entry[0])
        population.append(entry[1])

    print(column_names[0])
    print(geo, "\n")

    print(column_names[1])
    print(population)

except Exception as error:
    # print error message
    print(f"Error message: {error}")

    # print entry and index
    print(f"Entry: {entry}\t Line index: {population_list_by_line.index(line)}")
```

```
Error message: list index out of range
Entry: ['']  Line index: 29
```

The error message indicates that one of the list operations using the slice operator is accessing an invalid index. With a slight adjustment, the code block also reveals the cause of the error.

If the empty line is removed from the list, splitting the country names and the values works correctly.

```
# remove empty line
population_list_by_line.remove('')

# Initialize empty lists before the loop
countries = []
population_numbers = []

try:
    for line in population_list_by_line:
        entry = line.split(',')
```

```
        countries.append(entry[0])
        population_numbers.append(entry[1])

    print(column_names[0])
    print(countries, "\n")

    print(column_names[1])
    print(population_numbers)

except IndexError as error:
    print(error)
```

```
GEO
['Belgien', 'Bulgarien', 'Tschechien', 'Daenemark', 'Deutschland einschliesslich ehemalige DI

Value
['11467923', '7000039', '10528984', '5799763', '82940663', '1324820', '4904240', '10722287',
1', '9772756', '493559', '17423013', '8842000', '37972812', '10276617', '19405156', '2080908
```

7.7 Task: Interpret Data

1. Determine the minimum and maximum population and the corresponding countries.
2. Clean any incorrect values if necessary.
3. How many people live in Europe in total?

- What is the data type of the `population` list?
- What is the data type of the entries in the `population` list?

Sample Solution: Full File Reading

```
# 1. Determine the minimum and maximum population values and the corresponding countries
population_list = []
for element in population:
     population_list.append(int(element))

# Convert the population values from strings to integers
population = [int(number) for number in population]

# 2. Clean errors
# Both the error in the population list and the corresponding country are removed.
for i in range(len(population) - 1, -1, -1):  # Iterate backwards so removing elements does
    if population[i] < 0:
        population.pop(i)
        geo.pop(i)

for i in range(len(population)):
    if population[i] == max(population):
        maximum = i
    elif population[i] == min(population):
        minimum = i

maximum_country = geo[maximum]
minimum_country = geo[minimum]
print(f"Maximum: {maximum_country} with {max(population)} inhabitants")
print(f"Minimum: {minimum_country} with {min(population)} inhabitants")

# 3. Total population of Europe
total_population = sum(population)  # Uganda is not part of Europe but has already been rem
print(f"The total population of Europe is {total_population}")

# 4.
print(f"The list 'population' has the data type {type(population)}")
print(f"The entries in the list have the data type {type(population[0])}")
```

```
Maximum: Deutschland einschliesslich ehemalige DDR with 82940663 inhabitants
Minimum: Malta with 493559 inhabitants
The total population of Europe is 514117784
The list 'population' has the data type <class 'list'>
The entries in the list have the data type <class 'int'>
```

Solution template by Marc Sönnecken.

7.8 Reading as a List

A file object can also be read directly into a list. The method `file_object.readlines()` returns a list where each line is an entry. Similarly, the `list()` function can be applied to file objects. Both approaches return the same list, including the newline character `\n`.

```
file_path = "01-daten/einwohner_europa_2019.csv"
file_object_population = open(file_path, 'r')

# Using readlines method
population = file_object_population.readlines()
print(population)

## Reset file pointer
file_object_population.seek(0);

# Using list function
population = list(file_object_population)
print(population)

# Close the file
file_object_population.close();
```

```
['GEO,Value\n', 'Belgien,11467923\n', 'Bulgarien,7000039\n', 'Tschechien,10528984\n', 'Daener
1\n', 'Ungarn,9772756\n', 'Malta,493559\n', 'Niederlande,17423013\n', 'Oesterreich,8842000\n
['GEO,Value\n', 'Belgien,11467923\n', 'Bulgarien,7000039\n', 'Tschechien,10528984\n', 'Daener
1\n', 'Ungarn,9772756\n', 'Malta,493559\n', 'Niederlande,17423013\n', 'Oesterreich,8842000\n
```

To remove the line break, you could use the slice operator to remove the last character of each list entry.

Another option is the `str.replace(old, new, count=-1)` method, which can be used to replace or delete characters. The parameters `old` and `new` specify the string to be replaced and the string to insert, and *must be passed positionally*. The `count` parameter controls how many occurrences of `old` are replaced. By default, every occurrence is replaced.

```
print('Dog'.replace('Do', 'Mu'))

string_value = 'A  short text without  double spaces.'

print(string_value.replace('  ', ' '))
```

```
Mug
A short text without double spaces.
```

The `str.replace()` method can also be used for deletion. If an empty string is passed as the `new` parameter, the substring given in `old` will be removed.

```
print(string_value.replace(' ', '').replace('double', ''))
```

```
Ashorttextwithoutspaces.
```

The `str.replace()` method can be used to clean the newline characters from the read-in list.

```
file_path = "01-daten/einwohner_europa_2019.csv"
population_file = open(file_path, 'r')

# Using readlines method
population = population_file.readlines()
population_cleaned = []

for item in population:
    population_cleaned.append(item.replace('\n', ''))

population = population_cleaned
print(population)

# Close the file
population_file.close()
```

```
['GEO,Value', 'Belgien,11467923', 'Bulgarien,7000039', 'Tschechien,10528984', 'Daenemark,5799
1', 'Ungarn,9772756', 'Malta,493559', 'Niederlande,17423013', 'Oesterreich,8842000', 'Polen,3
```

7.9 Writing Files

To write files, you need to use the `write` method of a file object. This method takes the string you want to write as an argument.

```
file_path = "01-daten/neue_datei.txt"

# Open file for writing
file = open(file_path, mode='w')

# Write content to the file
file.write("Procrastination at Universities\n\n".upper())
file.write("CHAPTER 1: The Beginning is Always Hard\nPlaceholder: I will write the rest later

# Close the file
file.close()
```

The file can now be read.

```
file_content = open(file_path, mode='r')
text = file_content.read()
print(text)

file_content.close()
```

```
PROCRASTINATION AT UNIVERSITIES

CHAPTER 1: The Beginning is Always Hard
Placeholder: I will write the rest later.
```

7.10 Task: Writing files

1. Create a new file with the extension `.txt` and name it after your hometown. Write 10 lines of information about the city in this file.

(Arnold (2023b))

8 Importing Modules and Packages

Python's functionality can be significantly extended by importing modules and packages. Modules and packages are libraries that contain function definitions.

> **!** Wichtig 1: Modules and Packages
>
> **Modules** Modules are files that contain function definitions.
> Modules are imported using the `import` keyword followed by their name, e.g., `import glob`.
>
> **Packages** Packages are collections of modules.
> Modules contained in packages are imported using the `import` keyword with the syntax package.module, e.g., `import matplotlib.pyplot`.

Modules and packages are loaded in Python using the `import` keyword.
For example, the module responsible for generating (pseudo-)random numbers, `random`, can be imported with `import random`. Afterwards, the functions of the module are available under the module name using the syntax `module.function()`.

```
import random

print(random.randint(1, 10)) # Random integer between 1 and 10
```

```
5
```

The Matplotlib package provides many functions for the graphical representation of data. The module `matplotlib.pyplot` provides an interface to the functions it contains.

```
import matplotlib.pyplot

random_data = []  # create an empty list
for i in range(10):
    random_number = random.randint(1, 10)
    random_data.append(random_number)

matplotlib.pyplot.plot(random_data)
```

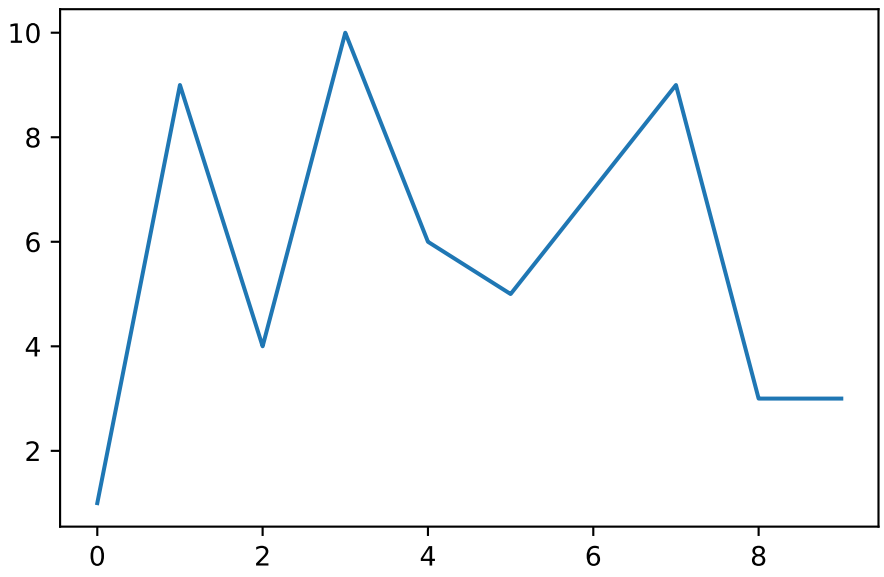


Abbildung 8.1: Plot using the pyplot module from the matplotlib package

Warning 7: Importing Functions Directly into the Namespace

In Python, it is also possible to import functions directly into the Python namespace, so they can be used without the `module.function()` syntax. This can be done using the `from` keyword.

```
from random import randint
print(f"The function randint is now available directly: {randint(1, 100)}")
```

```
The function randint is now available directly: 59
```

Through `from modulename import *`, it is even possible to import all functions from a module directly into Python's namespace. However, in general, directly importing functions or an entire module into Python's namespace should be avoided. On one hand, this risks namespace collisions—for example, the `sum()` function exists in base Python, in NumPy, and in Pandas. On the other hand, it makes the code less readable, since it is no longer clear which module a used function comes from.

8.1 import as

To simplify long module names, the `as` keyword can be used when importing to assign a new name to the module.

```
import matplotlib.pyplot as plt

plt.plot(random_data)
```

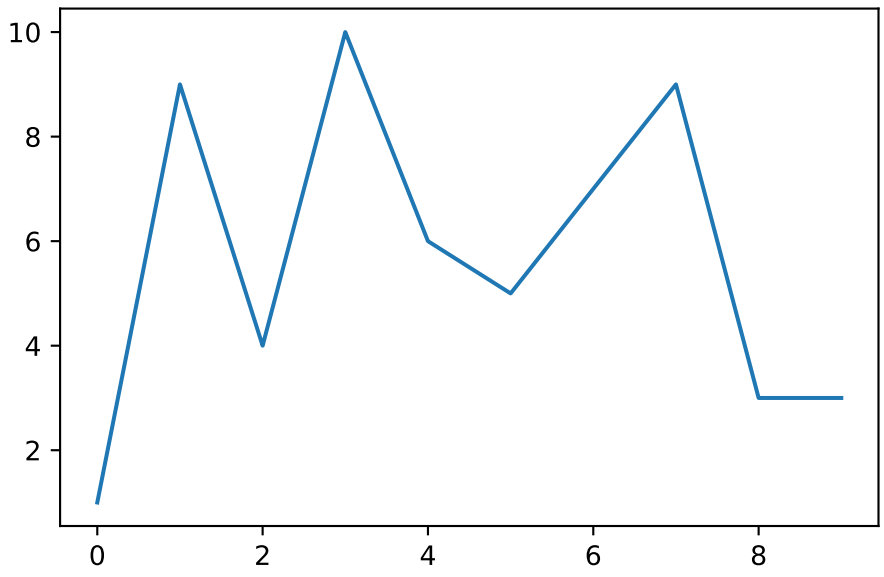


Abbildung 8.2: Plot using the pyplot module from the matplotlib package

For frequently used modules, certain abbreviations have become standard. In the exercises, the following packages and abbreviations are commonly used:

Module	Abbreviation	Command
NumPy	np	import numpy as np
Pandas	pd	import pandas as pd
matplotlib.pyplot	plt	import matplotlib.pyplot as plt

8.2 Small Module Overview

Since it is not possible to cover all these modules, only a few modules that might be relevant for the target audience of this script are listed below. *Note: The proper names of some modules use capitalization, e.g., the NumPy module. However, when importing the modules, the names are written in lowercase.* Therefore, capitalization is omitted in the following list.

- **math**: mathematical functions and constants
- **scipy**: scientific functions
- **sys**: interaction with the Python interpreter
- **os**: interaction with the operating system
- **glob**: searching through filesystem paths
- **multiprocessing / threading**: parallel programming with processes / threads
- **matplotlib**: data visualization and figure creation
- **numpy**: numerical operations and functions
- **pandas**: reading and analyzing data
- **time**: time functions

(Arnold (2023b))

9 The Essentials

Python is a very versatile programming language and therefore one of the most widely used. The Python base provides the fundamental framework for statistical programming. Specialized modules significantly extend its functionality. You can learn more about this in the video.

Quellen

Arnold, Simone (2023a). *Datenanalyse mit Python. Datentypen und Grundlagen.*
– (2023b). *Datenanalyse mit Python. Funktionen Module Dateien.*
– (2023c). *Datenanalyse mit Python. Idee des Kurses 'Datenanalyse mit Python'.*
– (2023d). *Datenanalyse mit Python. Schleifen und Abzweigungen.*
Matthes, Eric (2017). *Python Crashkurs: Eine praktische, projektbasierte Programmiereinführung.* 1. Aufl. Heidelberg: dpunkt.verlag. ISBN: 978-3-86490-444-8.